



**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN**

**ĐỒ ÁN MÔN HỌC
THỊ GIÁC MÁY TÍNH**

**TRÍCH XUẤT THÔNG TIN
BIÊN LAI TIẾNG VIỆT**

*So sánh kiến trúc Donut (OCR-free)
và VietOCR + LayoutXLM (OCR-based)*

Giảng viên hướng dẫn:

ThS. Nguyễn Đình Quý

Bộ môn:

Khoa học máy tính

Sinh viên thực hiện:

Trần Phùng Đức Anh - 0204168

Nguyễn Minh Năng - 0211668

Nguyễn Việt Hùng - 0208768

Lớp:

68CS2

Hà Nội, 2026

Lời cảm ơn

Lời đầu tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Nguyễn Đình Quý, giảng viên bộ môn Khoa học máy tính, Khoa Công nghệ thông tin, Trường Đại học Xây dựng Hà Nội. Thầy đã tận tình hướng dẫn, định hướng khoa học và cung cấp những chỉ dẫn vô cùng quý báu trong suốt quá trình nghiên cứu và hoàn thành đồ án môn học Thị giác máy tính này.

Chúng em cũng xin gửi lời cảm ơn đến các thầy cô giáo trong Khoa Công nghệ thông tin đã truyền đạt những kiến thức nền tảng bổ ích, tạo điều kiện thuận lợi nhất để chúng em thực hiện đề tài.

Cuối cùng, xin cảm ơn gia đình, bạn bè và tập thể lớp 68CS2 đã luôn động viên, hỗ trợ và đóng góp ý kiến để nhóm hoàn thiện báo cáo đồ án này một cách tốt nhất. Do kiến thức và kinh nghiệm thực tế còn hạn chế, báo cáo không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự đóng góp và chỉ bảo của quý thầy cô.

Chúng em xin chân thành cảm ơn!

Hà Nội, năm 2026

Nhóm sinh viên thực hiện

Contents

Lời cảm ơn	1
Danh mục hình ảnh	6
Danh mục bảng biểu	7
Danh mục từ viết tắt	8
Danh mục từ viết tắt	8
Tóm tắt	9
1 Tổng quan đề tài	10
1.1 Đặt vấn đề	10
1.2 Mục tiêu đề tài	10
1.3 Phạm vi đề tài	11
1.4 Đóng góp chính	11
1.5 Cấu trúc báo cáo	11
2 Cơ sở lý thuyết	12
2.1 Bài toán trích xuất thông tin tài liệu	12
2.2 Nhận dạng ký tự quang học	12
2.2.1 Tổng quan về OCR	13
2.2.2 PaddleOCR — Phát hiện văn bản	13
2.2.3 VietOCR — Nhận dạng văn bản tiếng Việt	14
2.3 Mô hình Donut	15
2.3.1 Kiến trúc tổng quan	15
2.3.2 Vision Encoder: Swin Transformer	16
2.3.3 Text Decoder: mBART	16
2.3.4 Ưu điểm và hạn chế	17
2.4 Mô hình LayoutXLM	17
2.4.1 Kiến trúc tổng quan	18
2.4.2 Những đa phương thức	18
2.4.3 Spatial-aware Self-Attention	19
2.4.4 Token Classification với BIO tagging	20
2.4.5 Ưu điểm và hạn chế	20
2.5 Các độ đo đánh giá	21
2.5.1 Exact Match (EM)	21

2.5.2	Normalized Edit Similarity (NES)	21
2.5.3	Character Error Rate (CER)	22
2.5.4	End-to-End Latency	22
2.6	Các công trình liên quan	22
3	Thu thập và xây dựng tập dữ liệu	24
3.1	Nguồn dữ liệu	24
3.1.1	MC-OCR 2021	24
3.1.2	Dữ liệu tự thu thập	24
3.2	Hướng dẫn gán nhãn (Annotation Guideline)	25
3.3	Schema dữ liệu thống nhất (Unified JSONL)	26
3.4	Chiến lược chia dữ liệu (Data Split)	27
3.4.1	Main Split	27
3.4.2	Thông kê tập dữ liệu	27
3.5	Kiểm tra dữ liệu (Validation)	27
4	Tiền xử lý dữ liệu	29
4.1	Tiền xử lý ảnh	29
4.1.1	Resize	29
4.1.2	Nâng cao chất lượng ảnh	29
4.2	Tăng cường dữ liệu (Data Augmentation)	29
4.3	Chuẩn hoá văn bản (Text Normalization)	30
4.3.1	Chuẩn hoá chung	30
4.3.2	Chuẩn hoá theo trường	30
4.4	Xây dựng OCR Cache	30
4.5	Thí nghiệm Ablation tiền xử lý ảnh cho OCR	31
5	Phương pháp thực hiện	33
5.1	Tổng quan kiến trúc hệ thống	33
5.2	Baseline: OCR + Trích xuất dựa trên luật (Rule-based)	34
5.2.1	Pipeline tổng quan	34
5.2.2	Quy tắc trích xuất cụ thể	34
5.3	Donut: End-to-End OCR-free	35
5.3.1	Chuẩn bị dữ liệu	35
5.3.2	Cấu hình huấn luyện	36
5.3.3	Suy luận (Inference)	36
5.4	VietOCR + LayoutXLM: OCR-based Layout-aware	37
5.4.1	Chuẩn bị dữ liệu	37
5.4.2	Cấu hình huấn luyện	38
5.4.3	Suy luận và hậu xử lý (Inference & Post-processing)	38

5.5	Hậu xử lý chung (Post-processing)	40
6	Thực nghiệm và huấn luyện mô hình	41
6.1	Môi trường thực nghiệm	41
6.2	Điều kiện so sánh công bằng	41
6.3	Quá trình huấn luyện DONUT	42
6.4	Quá trình huấn luyện LAYOUTXLM	43
6.5	Pipeline thực nghiệm tự động	44
7	Kết quả thực nghiệm và đánh giá	45
7.1	Bảng kết quả chính	45
7.1.1	Exact Match (EM)	45
7.1.2	Normalized Edit Similarity (NES)	45
7.1.3	Character Error Rate (CER)	45
7.2	Biểu đồ so sánh	46
7.3	Phân tích kết quả theo từng trường	46
7.3.1	Trường store_name	46
7.3.2	Trường date	47
7.3.3	Trường total	47
7.3.4	Trường address	48
7.4	Phân tích lỗi (Error Analysis)	48
7.4.1	Phân loại lỗi (Error Taxonomy)	48
7.4.2	Phân bố lỗi	48
7.4.3	Nhận xét	49
7.5	Bảng xếp hạng tổng hợp	50
7.6	Phân tích bổ sung và định hướng kiểm chứng	50
7.6.1	Định hướng thực nghiệm Ablation cho tiền xử lý OCR	50
7.6.2	Phân tích cận trên lý thuyết thông qua Oracle OCR	50
7.7	Demo ứng dụng (Gradio)	51
8	Kết luận và hướng phát triển	52
8.1	Kết luận	52
8.2	Hạn chế	52
8.3	Hướng phát triển	52
	Tài liệu tham khảo	55
A	Mẫu dữ liệu JSONL thống nhất	56
B	Cấu trúc mã nguồn	57

C	Cấu hình huấn luyện đầy đủ	58
C.1	Cấu hình DONUT (<code>donut.yaml</code>)	58
C.2	Cấu hình LAYOUTXLM (<code>layoutxlm.yaml</code>)	59
C.3	Cấu hình luật heuristic (<code>baseline.yaml</code>)	60
D	Mẫu kết quả trích xuất	62
E	Giao diện Demo Gradio	63
E.1	Hướng dẫn cài đặt và chạy demo	63

List of Figures

2.1	Kiến trúc tổng quan mô hình DONUT [5]. Ảnh tài liệu được mã hóa bởi Swin Transformer, sau đó giải mã thành chuỗi JSON bởi mBART Decoder.	15
2.2	Kiến trúc mô hình LAYOUTXLM [16]. Mô hình kết hợp ba loại thông tin: văn bản, bố cục không gian 2D và đặc trưng thị giác thông qua cơ chế nhúng đa phương thức.	18
3.1	Ví dụ một số ảnh biên lai trong tập dữ liệu tự thu thập	25
5.1	Sơ đồ tổng quan kiến trúc ba pipeline trích xuất thông tin biên lai.	33
5.2	Ví dụ gán nhãn Begin-Inside-Outside — Lược đồ gán nhãn chuỗi (BIO) cho các từ trên biên lai.	38
6.1	Đường cong loss huấn luyện và validation của DONUT theo epoch.	42
6.2	Đường cong loss huấn luyện và validation của LAYOUTXLM theo epoch.	43
7.1	Biểu đồ so sánh <i>Exact Match</i> (%) giữa ba phương pháp theo từng trường thông tin.	46
7.2	Biểu đồ so sánh <i>Normalized Edit Similarity</i> (%) giữa ba phương pháp theo từng trường thông tin.	47
7.3	Phân bố các loại lỗi theo phương pháp. Mã lỗi tham chiếu Bảng 7.4.	49
7.4	Giao diện demo ứng dụng trích xuất thông tin biên lai trên nền tảng Gradio. . .	51
E.1	Giao diện ứng dụng demo Gradio cho trích xuất thông tin biên lai	63

List of Tables

1.1	Các trường thông tin cần trích xuất từ biên lai.	11
2.1	Hệ thống nhãn BIO cho bài toán trích xuất thông tin biên lai.	20
2.2	Tổng hợp các độ đo đánh giá được sử dụng trong đồ án.	22
3.1	Ánh xạ nhãn từ MC-OCR 2021 sang schema đồ án	24
3.2	Các trường trong schema JSONL thống nhất	26
3.3	Phân bố số lượng mẫu theo tập dữ liệu	28
3.4	Tỷ lệ thiếu nhãn theo từng trường	28
4.1	Cấu hình tăng cường dữ liệu	30
4.2	Ví dụ chuẩn hoá văn bản theo trường	31
4.3	Kết quả thí nghiệm ablation tiền xử lý ảnh cho OCR	32
5.1	Quy tắc trích xuất cho từng trường thông tin trong pipeline baseline.	35
5.2	Cấu hình huấn luyện mô hình DONUT.	36
5.3	Cấu hình huấn luyện mô hình LAYOUTXLM.	39
5.4	Danh sách 9 nhãn BIO sử dụng trong mô hình LAYOUTXLM.	39
6.1	So sánh siêu tham số huấn luyện giữa DONUT và LAYOUTXLM.	41
7.1	Kết quả <i>Exact Match</i> (%) trên tập test (234 mẫu). In đậm: giá trị tốt nhất theo cột.	45
7.2	Kết quả <i>Normalized Edit Similarity</i> (%) trên tập test (234 mẫu). In đậm: giá trị tốt nhất theo cột.	45
7.3	Kết quả <i>Character Error Rate</i> (%) trên tập test (234 mẫu). In đậm: giá trị tốt nhất (thấp nhất) theo cột. *CER > 100%: dự đoán dài hơn ground-truth (hallucination).	46
7.4	Bảng phân loại lỗi (Error Taxonomy) trong trích xuất thông tin biên lai.	48
7.5	Bảng xếp hạng tổng hợp giữa ba phương pháp.	50
D.1	Mẫu kết quả trích xuất so với nhãn thật	62

Danh mục từ viết tắt

BART Bidirectional and Auto-Regressive Transformers. 16, 33, 35

BIO Begin-Inside-Outside — Lược đồ gán nhãn chuỗi. 6, 7, 20, 33, 37–39, 48, 50

CER Character Error Rate — Tỷ lệ lỗi ký tự. 22, 41, 45

CNN Convolutional Neural Network — Mạng nơ-ron tích chập. 19

CTC Connectionist Temporal Classification — Phân loại thời gian kết nối. 14

EM Exact Match — Khớp chính xác. 21, 22, 41, 45

IE Information Extraction — Trích xuất thông tin. 10, 11

IoU Intersection over Union — Tỷ lệ giao trên hợp. 37

KIE Key Information Extraction — Trích xuất thông tin then chốt. 12

MC-OCR Mobile-Captured OCR — Nhận dạng ký tự chụp di động. 11, 24, 41

MSA Multi-head Self-Attention — Tự chú ý đa đầu. 16, 23

NER Named Entity Recognition — Nhận dạng thực thể có tên. 10, 12

NES Normalized Edit Similarity — Độ tương đồng biên tập chuẩn hoá. 21, 22, 41, 45

NFC Unicode Normalization Form Composed — Dạng chuẩn hoá Unicode tổ hợp. 11, 30, 40, 41, 48

OCR Optical Character Recognition — Nhận dạng ký tự quang học. 10–13, 15, 17, 21, 22, 29, 31, 33–35, 37, 44, 45, 48, 52, 53

RoI Region of Interest — Vùng quan tâm. 19

Tóm tắt

Trong những năm gần đây, nhu cầu chuyển đổi số và tự động hóa quy trình quản lý tài chính doanh nghiệp cũng như cá nhân ngày càng tăng cao, đặt ra bài toán số hóa và trích xuất thông tin tự động từ các tài liệu hóa đơn, biên lai bán lẻ. Đối với ngôn ngữ tiếng Việt, việc trích xuất thông tin từ biên lai gặp nhiều thách thức lớn do chất lượng ảnh chụp từ thiết bị di động không đồng đều, cấu trúc biên lai đa dạng và đặc thù ngôn ngữ có dấu dễ bị nhận dạng sai. Đồ án này tập trung nghiên cứu và xây dựng hệ thống trích xuất thông tin tự động bốn trường dữ liệu quan trọng từ biên lai tiếng Việt bao gồm: tên cửa hàng (**store_name**), ngày tháng (**date**), tổng tiền thanh toán (**total**) và địa chỉ (**address**). Chúng em tiến hành thực nghiệm và so sánh chi tiết ba phương pháp tiếp cận tiêu biểu: (1) Phương pháp heuristics dựa trên luật kết hợp OCR truyền thống (BASELINE), (2) Mô hình học sâu End-to-End không cần OCR (DONUT), và (3) Mô hình học sâu đa phương thức tích hợp thông tin văn bản, bố cục và hình ảnh (LAYOUTXLM) kết hợp với công cụ nhận dạng văn bản tiếng Việt VIETOCR. Các mô hình được huấn luyện và đánh giá trên bộ dữ liệu thống nhất gồm 1.559 mẫu biên lai thu thập từ cuộc thi MC-OCR 2021 kết hợp với dữ liệu tự chụp thực tế. Kết quả thực nghiệm cho thấy mô hình LAYOUTXLM đạt hiệu năng vượt trội với độ chính xác tuyệt đối Macro EM đạt 43,48%, tương tự độ tương đồng văn bản Macro NES đạt 61,86% và tỷ lệ lỗi ký tự Macro CER đạt 39,64%. Trong khi đó, phương pháp BASELINE cho thấy ưu thế vượt trội ở trường thông tin ngày tháng nhờ luật biểu thức chính quy (EM đạt 74,79%), còn mô hình DONUT gặp hạn chế lớn về hiện tượng sinh ảo (hallucination) do giới hạn về kích thước tập dữ liệu huấn luyện. Cuối cùng, nhóm nghiên cứu đã xây dựng thành công ứng dụng demo tương tác trực quan bằng giao diện GRADIO phục vụ việc kiểm thử thực tế.

Từ khoá: trích xuất thông tin, biên lai, Donut, LayoutXLM, OCR, tiếng Việt.

1. Tổng quan đề tài

Chương này sẽ giới thiệu khái quát về đề tài nghiên cứu, nêu bật tính cấp thiết của việc số hóa tài liệu hóa đơn tại Việt Nam, đồng thời làm rõ mục tiêu, phạm vi và những đóng góp khoa học chính mà đề án đã đạt được. Cuối cùng, cấu trúc tổng quát của toàn bộ báo cáo cũng sẽ được tóm tắt chi tiết ở phần cuối chương.

1.1. Đặt vấn đề

Với sự phát triển mạnh mẽ của kinh tế số và thương mại điện tử, hàng triệu giao dịch mua sắm diễn ra mỗi ngày tại các siêu thị, cửa hàng tiện lợi và các hộ kinh doanh cá thể tại Việt Nam. Lượng biên lai, hóa đơn bán lẻ phát sinh là vô cùng khổng lồ. Việc nhập liệu, đối soát thủ công các thông tin này vào các phần mềm kế toán không chỉ gây mất nhiều thời gian, chi phí nhân sự mà còn dễ xảy ra sai sót không đáng có. Do đó, nhu cầu tự động hóa quy trình số hóa hóa đơn bằng kỹ thuật nhận dạng ký tự quang học ([Optical Character Recognition — Nhận dạng ký tự quang học \(OCR\)](#)) và trích xuất thông tin thực thể liên kết ([Information Extraction — Trích xuất thông tin \(IE\)](#)) đang trở thành một đòi hỏi cấp thiết.

Tuy nhiên, bài toán trích xuất thông tin biên lai tại Việt Nam vẫn đối mặt với nhiều thách thức lớn từ thực tế. Ảnh đầu vào thường được chụp bởi người dùng bằng thiết bị di động cá nhân, dẫn đến hiện tượng ảnh bị mờ, nghiêng, thiếu sáng hoặc bị phản quang. Bên cạnh đó, các biên lai tiếng Việt có bố cục cực kỳ đa dạng, font chữ không đồng nhất, giấy in nhiệt dễ bị mờ theo thời gian. Đặc biệt, đặc trưng tiếng Việt với hệ thống dấu thanh phức tạp đặt ra yêu cầu rất cao cho hệ thống nhận dạng, nhằm tránh các lỗi dịch nghĩa do sai lệch ký tự.

1.2. Mục tiêu đề tài

Mục tiêu cốt lõi của đề án là thiết kế và hiện thực hóa một giải pháp trích xuất tự động các trường thông tin quan trọng từ ảnh chụp biên lai tiếng Việt một cách hiệu quả và chính xác, đáp ứng các tiêu chuẩn ứng dụng thực tế.

Đề tài tập trung trích xuất 4 trường thông tin then chốt từ ảnh biên lai tiếng Việt, đồng thời so sánh có hệ thống ba hướng tiếp cận:

- **DONUT [5]** — kiến trúc OCR-free, sinh trực tiếp văn bản từ ảnh mà không cần bước OCR riêng biệt.
- **VIETOCR + LAYOUTXLM [16]** — pipeline OCR-based: nhận dạng văn bản trước, sau đó gán nhãn chuỗi ([Named Entity Recognition — Nhận dạng thực thể có tên \(NER\)](#)).
- **Baseline rule-based** — phương pháp đối sánh từ khoá và biểu thức chính quy (regex), dùng làm cơ sở so sánh.

[Bảng 1.1](#) mô tả chi tiết 4 trường thông tin cần trích xuất.

Table 1.1: Các trường thông tin cần trích xuất từ biên lai.

Trường	Định dạng
store_name	Chuỗi Unicode Unicode Normalization Form Composed — Dạng chuẩn hoá Unicode tổ
date	YYYY-MM-DD
total	Chuỗi số nguyên
address	Chuỗi Unicode NFC

1.3. Phạm vi đề tài

Để giải quyết bài toán một cách khả thi và tập trung sâu vào các công nghệ cốt lõi, phạm vi nghiên cứu của đồ án được giới hạn cụ thể như sau:

- **Đầu vào:** Ảnh biên lai chụp bằng điện thoại (định dạng JPG/PNG).
- **Đầu ra:** Tập JSON chứa 4 trường: **store_name**, **date**, **total**, **address**.
- **Ngoài phạm vi:** danh sách sản phẩm, mã số thuế (VAT), số điện thoại, biên lai dạng PDF.

1.4. Đóng góp chính

Đồ án đã mang lại một số đóng góp thực tiễn cho bài toán xử lý tài liệu tiếng Việt:

1. **Bộ dữ liệu kết hợp:** Ghép nối bộ dữ liệu [Mobile-Captured OCR — Nhận dạng ký tự chụp di động \(MC-OCR\)](#) 2021 với dữ liệu tự thu thập, tổng cộng khoảng 1559 mẫu biên lai tiếng Việt đã gán nhãn.
2. **So sánh có hệ thống 3 phương pháp:** DONUT (OCR-free), VIETOCR + LAYOUTXLM (OCR-based), và Baseline rule-based; đánh giá trên cùng tập dữ liệu với các chỉ số *EM* và *NES*.
3. **Phân loại lỗi chi tiết:** Đề xuất bảng phân loại 11 loại lỗi (error taxonomy) giúp phân tích nguyên nhân sai sót của từng phương pháp.
4. **Demo ứng dụng web:** Xây dựng giao diện trình diễn bằng Gradio, cho phép tải ảnh biên lai lên và xem kết quả trích xuất trực tiếp.

1.5. Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức như sau:

- **Mục 2** trình bày cơ sở lý thuyết về [OCR](#), [IE](#), và các kiến trúc mô hình được sử dụng.
- **Mục 3** mô tả bộ dữ liệu, quy trình thu thập và gán nhãn.
- **Mục 4** trình bày các bước tiền xử lý dữ liệu ảnh và văn bản.
- **Mục 5** chi tiết phương pháp huấn luyện và triển khai từng mô hình.
- **Mục 6** mô tả thiết lập thí nghiệm, siêu tham số và môi trường chạy.
- **Mục 7** trình bày và thảo luận kết quả thí nghiệm.
- **Mục 8** tổng kết và đề xuất hướng phát triển.

2. Cơ sở lý thuyết

Chương này trình bày các cơ sở lý thuyết nền tảng phục vụ cho đề án, bao gồm bài toán trích xuất thông tin tài liệu (Document Information Extraction - DIE), kỹ thuật nhận dạng ký tự quang học (OCR), các kiến trúc mô hình học sâu hiện đại như DONUT và LAYOUTXML, cùng các độ đo đánh giá và công trình liên quan được sử dụng để đối sánh thực nghiệm.

2.1. Bài toán trích xuất thông tin tài liệu

Trích xuất thông tin tài liệu (Document Information Extraction) là bài toán tự động nhận dạng và trích xuất các thông tin có cấu trúc từ hình ảnh tài liệu phi cấu trúc hoặc bán cấu trúc. Trong ngữ cảnh biên lai, bài toán **Key Information Extraction — Trích xuất thông tin then chốt (KIE)** yêu cầu xác định các trường thông tin then chốt như **store_name, date, address, total** từ ảnh chụp biên lai.

Về mặt toán học, đầu vào của bài toán trích xuất thông tin thực thể liên kết (KIE) là một tài liệu ảnh biên lai dạng bán cấu trúc $I \in \mathbb{R}^{H \times W \times C}$. Đầu ra mong muốn là một tập hợp các thực thể thông tin có cấu trúc dưới dạng cặp khóa-giá trị: $Y = \{(K_1, V_1), (K_2, V_2), \dots, (K_M, V_M)\}$, trong đó mỗi K_j tương ứng với một trong các trường cần trích xuất (ví dụ: **store_name, date, total, address**) và V_j là giá trị văn bản đã chuẩn hóa được trích xuất từ tài liệu. Quá trình này đòi hỏi mô hình không chỉ nhận diện đúng ký tự mà còn phải hiểu được quan hệ không gian và ngữ cảnh ngữ nghĩa giữa các khối văn bản để liên kết chúng thành các cặp khóa-giá trị chính xác.

Các hướng tiếp cận chính cho bài toán KIE có thể phân loại thành bốn nhóm:

1. **Dựa trên quy tắc (Rule-based):** Sử dụng biểu thức chính quy (regex) và các quy tắc thủ công để trích xuất thông tin từ văn bản đã OCR. Ưu điểm là đơn giản, dễ triển khai; nhược điểm là khó mở rộng và thiếu khả năng tổng quát hóa với các mẫu biên lai mới.
2. **OCR kết hợp NLP (OCR + NLP):** Sử dụng OCR để nhận dạng văn bản, sau đó áp dụng các kỹ thuật xử lý ngôn ngữ tự nhiên như NER để phân loại thực thể. Hướng này chỉ khai thác thông tin ngữ nghĩa mà bỏ qua bố cục không gian (layout) của tài liệu.
3. **OCR kết hợp mô hình nhận biết bố cục (OCR + Layout-aware):** Kết hợp OCR với các mô hình có khả năng hiểu bố cục không gian như LAYOUTLM [15], LAYOUTXML [16]. Đây là hướng tiếp cận được sử dụng cho pipeline OCR-based trong đề án này.
4. **End-to-End không cần OCR (OCR-free):** Các mô hình như DONUT [5] trực tiếp ánh xạ từ ảnh tài liệu sang chuỗi văn bản có cấu trúc, không cần module OCR riêng biệt. Ưu điểm là giảm phụ thuộc vào chất lượng OCR và đơn giản hóa pipeline.

2.2. Nhận dạng ký tự quang học

OCR là quá trình chuyển đổi hình ảnh chứa văn bản thành dạng văn bản có thể xử lý bằng máy tính. Trong pipeline OCR-based của đề án, OCR đóng vai trò quan trọng ở giai đoạn tiền xử lý

dữ liệu đầu vào cho mô hình LAYOUTXLM.

2.2.1. Tổng quan về OCR

Hệ thống OCR hiện đại thường gồm hai giai đoạn nối tiếp:

1. **Phát hiện văn bản (Text Detection):** Xác định vị trí các vùng chứa văn bản trong ảnh, thường biểu diễn dưới dạng hộp bao (bounding box) hoặc đa giác.
2. **Nhận dạng văn bản (Text Recognition):** Chuyển đổi vùng ảnh đã phát hiện thành chuỗi ký tự tương ứng.

Quy trình OCR hai giai đoạn diễn ra như sau: Đầu tiên, mô hình phát hiện văn bản (Text Detection) quét qua toàn bộ ảnh biên lai để xác định vị trí của từng dòng hoặc từng từ văn bản và trả về tọa độ các hộp bao (bounding boxes). Sau đó, ảnh nằm trong các vùng hộp bao này được cắt ra (crop) và truyền qua mô hình nhận dạng văn bản (Text Recognition) để chuyển đổi các đặc trưng thị giác của chữ viết thành chuỗi ký tự Unicode có dấu. Kết quả cuối cùng là danh sách các đoạn văn bản kèm theo tọa độ địa lý của chúng trên ảnh.

2.2.2. PaddleOCR — Phát hiện văn bản

PaddleOCR [2] là bộ công cụ OCR mã nguồn mở được phát triển bởi Baidu, sử dụng thuật toán DB (Differentiable Binarization) cho giai đoạn phát hiện văn bản. DB cho phép tối ưu hóa ngưỡng nhị phân hóa trực tiếp thông qua lan truyền ngược (backpropagation), giúp cải thiện đáng kể khả năng phát hiện văn bản so với các phương pháp sử dụng ngưỡng cố định.

Thuật toán phát hiện văn bản DB (Differentiable Binarization) cải tiến thuật toán binarization truyền thống bằng cách tích hợp trực tiếp bước nhị phân hóa khả vi vào mạng nơ-ron tích chập. Cấu trúc mạng DB bao gồm: (1) Backbone (ví dụ: ResNet hoặc MobileNetV3) đóng vai trò trích xuất đặc trưng hình ảnh đa tỷ lệ; (2) Mạng kim tự tháp đặc trưng (FPN) kết hợp đặc trưng từ các lớp khác nhau để tạo ra bản đồ đặc trưng giàu thông tin ngữ cảnh; (3) Đầu dự báo song song để sinh ra Probability Map (bản đồ xác suất chứa ký tự) và Threshold Map (bản đồ ngưỡng thích ứng). Quá trình nhị phân hóa khả vi thực hiện phép toán:

$$\hat{B}_{i,j} = \frac{1}{1 + \exp(-k(P_{i,j} - T_{i,j}))} \quad (2.1)$$

ở đây $P_{i,j}$ là xác suất tại điểm ảnh (i, j) , $T_{i,j}$ là ngưỡng tại điểm đó và k là hệ số phóng đại (thường được đặt mặc định là 50). Nhờ tính khả vi của hàm này, mạng có thể tối ưu hóa trực tiếp cả bản đồ xác suất lẫn bản đồ ngưỡng thích ứng thông qua lan truyền ngược, giúp việc phát hiện văn bản có độ chính xác cao hơn, đặc biệt là ở biên của các chữ viết.

Ưu điểm chính của PaddleOCR trong đồ án:

- Hỗ trợ đa ngôn ngữ, bao gồm tiếng Việt.
- Mô hình nhẹ (lightweight), phù hợp triển khai thực tế.

- Trả về tọa độ bounding box chính xác cho từng dòng văn bản, cung cấp thông tin không gian cần thiết cho LAYOUTXML.

2.2.3. VietOCR — Nhận dạng văn bản tiếng Việt

VietOCR [11] là mô hình nhận dạng văn bản được thiết kế đặc biệt cho tiếng Việt, sử dụng kiến trúc kết hợp VGG làm bộ trích xuất đặc trưng (feature extractor) và Transformer làm bộ giải mã chuỗi (sequence decoder).

Mô hình nhận dạng văn bản tiếng Việt VIETOCR sử dụng một bộ trích xuất đặc trưng dựa trên kiến trúc VGG-19 được sửa đổi. Vùng ảnh chứa dòng chữ sau khi cắt ra được đưa qua các lớp tích chập (convolutional layers) và lớp gộp tối đa (max-pooling layers) của VGG để giảm kích thước không gian và trích xuất vector đặc trưng visual. Cụ thể, ảnh đầu vào kích thước $H \times W \times C$ được biến đổi thành bản đồ đặc trưng có kích thước $H' \times W' \times D$ (với $H' \ll H$ và $W' \ll W$). Sau đó, đặc trưng visual này được làm phẳng theo chiều rộng để tạo thành chuỗi các vector đặc trưng đại diện cho các lát cắt thẳng đứng dọc theo dòng chữ, trước khi đưa vào bộ giải mã Transformer Encoder-Decoder tuần tự để ánh xạ thành văn bản.

Mô hình được huấn luyện với hàm mất mát [Connectionist Temporal Classification — Phân loại thời gian kết nối \(CTC\)](#) [3], cho phép huấn luyện mà không cần căn chỉnh (alignment) giữa ảnh đầu vào và nhãn chuỗi ký tự. Cụ thể, CTC định nghĩa xác suất phát ra ký tự π_t tại bước thời gian t như sau:

$$P(\pi_t | \mathbf{x}) = y_t^{\pi_t}, \quad \pi_t \in L' \quad (2.2)$$

trong đó $\mathbf{x}$ là chuỗi đặc trưng đầu vào, $y_t^{\pi_t}$ là xác suất đầu ra tại bước t ứng với ký tự π_t , và $L' = L \cup \{\text{blank}\}$ là bộ ký tự mở rộng thêm ký tự trống.

Xác suất của toàn bộ đường dẫn π có độ dài T được tính bằng tích các xác suất tại từng bước thời gian:

$$P(\pi | \mathbf{x}) = \prod_{t=1}^T y_t^{\pi_t} \quad (2.3)$$

Do nhiều đường dẫn khác nhau có thể ánh xạ về cùng một nhãn đầu ra $\mathbf{y}$ thông qua hàm rút gọn $\mathcal{B}$ (loại bỏ ký tự trống và các ký tự lặp liên tiếp), xác suất tổng của nhãn $\mathbf{y}$ được tính bằng tổng xác suất trên tất cả các đường dẫn hợp lệ:

$$P(\mathbf{y} | \mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} P(\pi | \mathbf{x}) \quad (2.4)$$

Hàm mất mát CTC được định nghĩa là logarit âm của xác suất này:

$$\mathcal{L}_{\text{CTC}} = -\ln P(\mathbf{y} | \mathbf{x}) \quad (2.5)$$

Quá trình tính toán $P(\mathbf{y} | \mathbf{x})$ sử dụng thuật toán quy hoạch động forward-backward, cho phép tính hiệu quả tổng trên tất cả các đường dẫn mà không cần liệt kê toàn bộ.

2.3. Mô hình Donut

DONUT (Document Understanding Transformer) [5] là mô hình hiểu tài liệu theo hướng end-to-end không cần OCR (OCR-free). Mô hình trực tiếp ánh xạ từ ảnh tài liệu sang chuỗi token có cấu trúc, loại bỏ hoàn toàn sự phụ thuộc vào module OCR bên ngoài.

2.3.1. Kiến trúc tổng quan

Kiến trúc DONUT gồm hai thành phần chính:

- **Vision Encoder:** Swin Transformer [10], mã hóa ảnh tài liệu thành chuỗi đặc trưng thị giác $\mathbf{z} = \{z_1, z_2, \dots, z_n\}$.
- **Text Decoder:** mBART [8], giải mã chuỗi đặc trưng thị giác thành chuỗi token đầu ra theo cơ chế tự hồi quy (autoregressive).

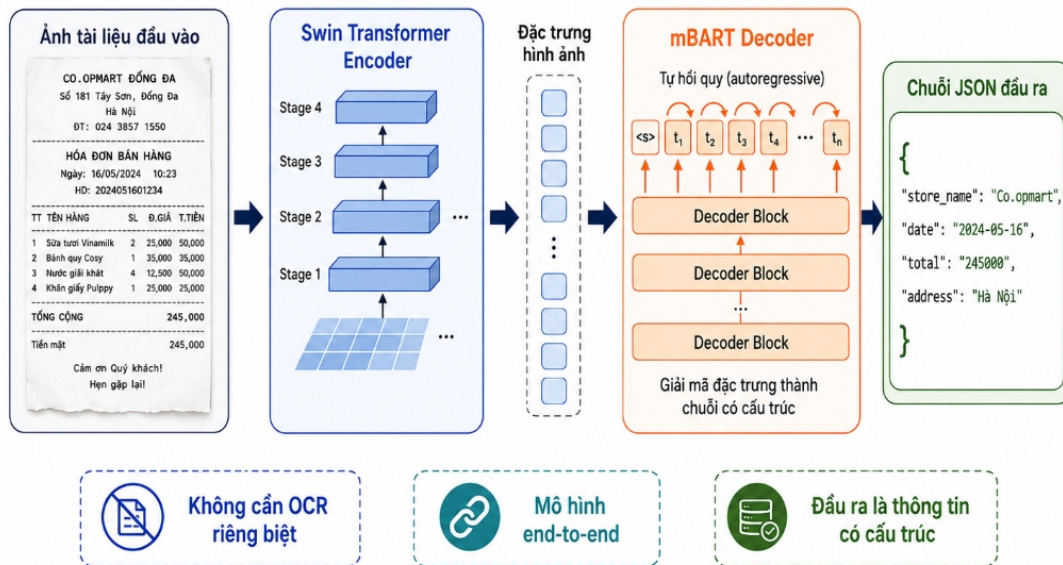


Figure 2.1: Kiến trúc tổng quan mô hình DONUT [5]. Ảnh tài liệu được mã hóa bởi Swin Transformer, sau đó giải mã thành chuỗi JSON bởi mBART Decoder.

2.3.2. Vision Encoder: Swin Transformer

Swin Transformer [10] là kiến trúc Transformer phân cấp cho thị giác máy tính, sử dụng cơ chế cửa sổ trượt (shifted window) để giảm độ phức tạp tính toán từ bậc hai xuống bậc tuyến tính theo kích thước ảnh.

Window-based Multi-head Self-Attention (W-MSA). Thay vì tính toán **Multi-head Self-Attention — Tự chú ý đa đầu (MSA)** trên toàn bộ ảnh, Swin Transformer chia ảnh thành các cửa sổ không chồng lấn có kích thước $M \times M$ pixel và thực hiện self-attention trong phạm vi từng cửa sổ. Cơ chế attention có dạng:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} + B \right) V \quad (2.6)$$

trong đó:

- $Q, K, V \in \mathbb{R}^{M^2 \times d}$ lần lượt là ma trận Query, Key và Value, được chiếu tuyến tính từ các đặc trưng đầu vào.
- d là số chiều của mỗi head attention.
- $B \in \mathbb{R}^{M^2 \times M^2}$ là ma trận thiên lệch vị trí tương đối (relative position bias), được học trong quá trình huấn luyện. Ma trận B giúp mô hình nhận biết vị trí tương đối giữa các token trong cùng cửa sổ mà không cần mã hóa vị trí tuyệt đối.

Shifted Window MSA (SW-MSA). Để tạo kết nối giữa các cửa sổ liền kề, ở lớp Transformer kế tiếp, các cửa sổ được dịch chuyển $\lfloor M/2 \rfloor$ pixel theo cả hai chiều ngang và dọc. Cơ chế W-MSA và SW-MSA được xen kẽ nhau qua các lớp Transformer, cho phép thông tin lan truyền giữa các cửa sổ.

Sự kết hợp giữa hai cơ chế W-MSA (Window Multi-head Self-Attention) và SW-MSA (Shifted Window Multi-head Self-Attention) giải quyết triệt để vấn đề mất liên kết giữa các cửa sổ không chồng lấn. Trong khi W-MSA thực hiện tính toán self-attention độc lập bên trong mỗi cửa sổ kích thước $M \times M$ để tối ưu hóa tốc độ, lớp SW-MSA kế tiếp sẽ dịch chuyển tọa độ phân chia cửa sổ đi một đoạn $\lfloor M/2 \rfloor$ pixel. Nhờ sự dịch chuyển này, các cửa sổ mới sẽ đè lên ranh giới của các cửa sổ cũ, cho phép các token ở ranh giới giao tiếp và truyền thông tin qua lại, duy trì sự nhất quán ngữ cảnh toàn cục của hình ảnh.

2.3.3. Text Decoder: mBART

Bộ giải mã sử dụng kiến trúc mBART [8], một biến thể đa ngôn ngữ của **Bidirectional and Auto-Regressive Transformers (BART)**. Tại mỗi bước giải mã i , decoder thực hiện hai cơ chế attention:

Self-Attention. Mỗi token s_i trong chuỗi đầu ra chú ý tới tất cả các token trước đó $s_{<i}$ (masked self-attention):

$$\mathbf{h}_i^{\text{self}} = \text{Self-Attention}(\mathbf{s}_i, \mathbf{s}_{<i}) \quad (2.7)$$

Cross-Attention. Kết quả self-attention $\mathbf{h}_i^{\text{self}}$ được sử dụng làm query để chú ý tới chuỗi đặc trưng thị giác $\mathbf{z}$ từ encoder:

$$\mathbf{h}_i^{\text{cross}} = \text{softmax} \left(\frac{(\mathbf{h}_i^{\text{self}} W^Q)(\mathbf{z} W^K)^T}{\sqrt{d}} \right) (\mathbf{z} W^V) \quad (2.8)$$

trong đó W^Q, W^K, W^V là các ma trận trọng số chiều tuyến tính cho Query, Key và Value tương ứng, và d là số chiều attention head.

Hàm mất mát huấn luyện. DONUT được huấn luyện bằng hàm cross-entropy trên chuỗi token đầu ra, với nhãn thực (teacher forcing) $y_{<i}^*$:

$$\mathcal{L}_{\text{Donut}} = - \sum_{i=1}^T \log P(y_i | y_{<i}^*, \mathbf{z}) \quad (2.9)$$

trong đó T là độ dài chuỗi đầu ra và $\mathbf{z}$ là đặc trưng thị giác từ Swin Encoder.

2.3.4. Ưu điểm và hạn chế

Ưu điểm.

- Loại bỏ hoàn toàn phụ thuộc vào module OCR, giảm lỗi tích lũy từ giai đoạn nhận dạng văn bản.
- Pipeline đơn giản, dễ huấn luyện và triển khai end-to-end.
- Kiến trúc tổng quát, có thể áp dụng cho nhiều bài toán hiểu tài liệu khác nhau bằng cách thay đổi prompt đầu vào.

Hạn chế.

- Yêu cầu dữ liệu huấn luyện lớn để đạt hiệu suất tốt.
- Tốc độ suy luận (inference) chậm hơn so với pipeline OCR-based do cơ chế giải mã tự hồi quy.
- Khó khăn khi xử lý ảnh có độ phân giải cao hoặc tài liệu nhiều trang.

2.4. Mô hình LayoutXLM

LAYOUTXLM [16] là mô hình tiền huấn luyện đa phương thức (multimodal) và đa ngôn ngữ (multilingual) cho bài toán hiểu tài liệu có bố cục phong phú. Mô hình kế thừa kiến trúc XLM-RoBERTa [1] và mở rộng bằng cách tích hợp thông tin văn bản, bố cục không gian 2D và đặc trưng thị giác.

2.4.1. Kiến trúc tổng quan

LayoutXLM là một mô hình học sâu đa phương thức được thiết kế để mở rộng khả năng hiểu tài liệu đa ngôn ngữ. Kiến trúc cốt lõi của LayoutXLM là Transformer Encoder, kế thừa trực tiếp từ XLM-RoBERTa nhưng tích hợp thêm các kênh thông tin về không gian 2D (bounding boxes) và đặc trưng thị giác (visual features) của từng vùng văn bản. Điều này giúp mô hình đồng thời biểu diễn được cả ngữ nghĩa của từ ngữ, vị trí vật lý của từ trên trang giấy và đặc điểm hình ảnh (font chữ, màu sắc, khung viền), giải quyết hiệu quả bài toán trích xuất thực thể từ tài liệu có cấu trúc bố cục phức tạp như hóa đơn.

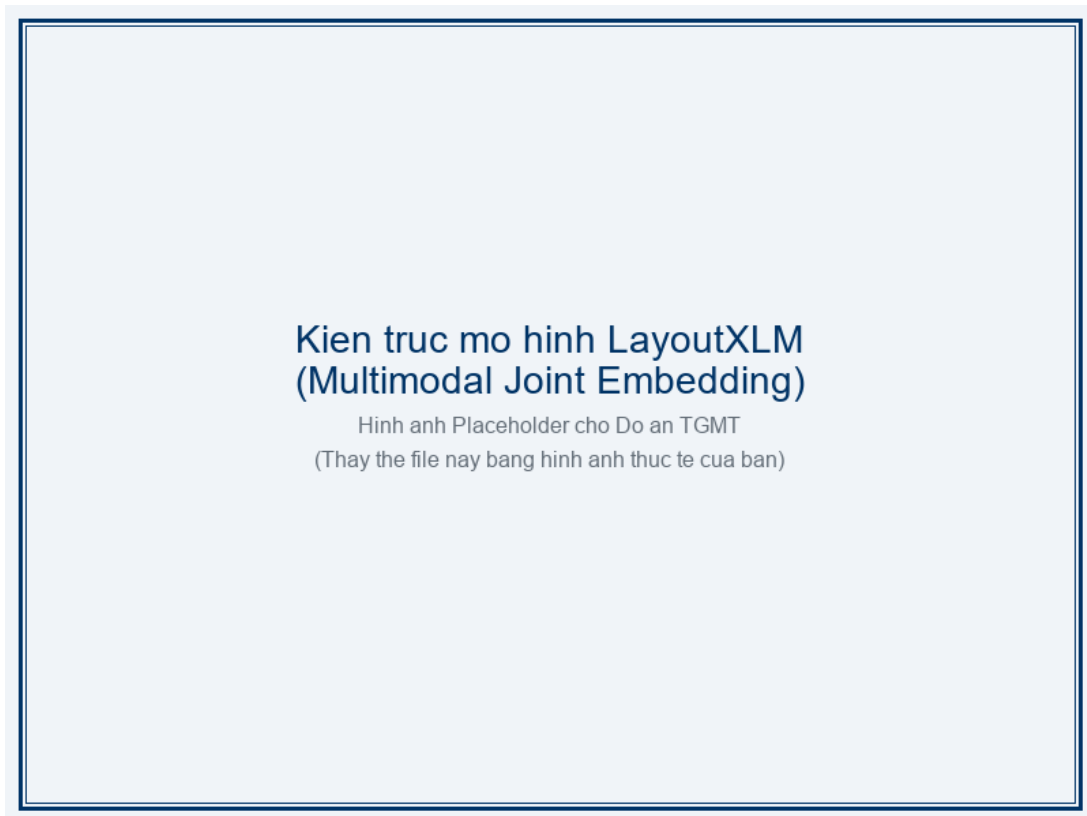


Figure 2.2: Kiến trúc mô hình LAYOUTXLM [16]. Mô hình kết hợp ba loại thông tin: văn bản, bố cục không gian 2D và đặc trưng thị giác thông qua cơ chế nhúng đa phương thức.

LAYOUTXLM sử dụng kiến trúc Transformer encoder với 12 lớp, kết hợp ba loại nhúng (embedding) khác nhau cho mỗi token đầu vào. Mô hình được tiền huấn luyện trên dữ liệu tài liệu đa ngôn ngữ với các tác vụ Masked Language Model (MLM), Text-Image Alignment (TIA) và Text-Image Matching (TIM).

2.4.2. Nhúng đa phương thức

Mỗi token w_i trong tài liệu được biểu diễn thông qua sự kết hợp của ba loại nhúng: nhúng văn bản, nhúng không gian 2D, và nhúng thị giác.

Nhúng văn bản (Text Embedding). Token w_i được ánh xạ thành vector nhúng thông qua bảng từ vựng đã học:

$$\mathbf{t}_i = \text{WordEmbedding}(w_i) \quad (2.10)$$

Nhúng không gian 2D (2D Spatial Embedding). Tọa độ bounding box $\text{bbox}_i = (x_1, y_1, x_2, y_2)$ của token được chuẩn hóa (normalize) về khoảng $[0, 1000]$ theo kích thước ảnh:

$$\hat{x} = \left\lfloor \frac{x}{W_{\text{img}}} \times 1000 \right\rfloor, \quad \hat{y} = \left\lfloor \frac{y}{H_{\text{img}}} \times 1000 \right\rfloor \quad (2.11)$$

trong đó W_{img} và H_{img} là chiều rộng và chiều cao của ảnh gốc. Chiều rộng w và chiều cao h của bounding box được tính là $w = x_2 - x_1$ và $h = y_2 - y_1$. Nhúng không gian được tính bằng tổng sáu thành phần nhúng:

$$\mathbf{e}_{2D}(\text{bbox}_i) = \mathbf{e}_{x_1}(x_1) + \mathbf{e}_{y_1}(y_1) + \mathbf{e}_{x_2}(x_2) + \mathbf{e}_{y_2}(y_2) + \mathbf{e}_w(w) + \mathbf{e}_h(h) \quad (2.12)$$

trong đó $\mathbf{e}_{x_1}, \mathbf{e}_{y_1}, \mathbf{e}_{x_2}, \mathbf{e}_{y_2}, \mathbf{e}_w, \mathbf{e}_h$ là các bảng nhúng riêng biệt cho từng thành phần tọa độ.

Nhúng thị giác (Visual Embedding). Đặc trưng thị giác của vùng ứng với token w_i được trích xuất bằng cơ chế [Region of Interest — Vùng quan tâm \(RoI\)](#) Align từ feature map của mạng [Convolutional Neural Network — Mạng nơ-ron tích chập \(CNN\)](#):

$$\mathbf{v}_i = \text{RoIAlign}(\text{FeatureMap}, \text{bbox}_i) W^V \quad (2.13)$$

trong đó W^V là ma trận chiếu tuyến tính đưa đặc trưng thị giác về cùng không gian với các nhúng khác.

Nhúng kết hợp. Ba loại nhúng được cộng element-wise tạo thành biểu diễn đầu vào cho mỗi token:

$$\mathbf{x}_i = \mathbf{t}_i + \mathbf{e}_{2D}(\text{bbox}_i) + \mathbf{v}_i \quad (2.14)$$

2.4.3. Spatial-aware Self-Attention

LAYOUTXLM sử dụng cơ chế self-attention có nhận biết không gian (spatial-aware), bổ sung thông tin thiên lệch vị trí 1D và 2D vào điểm attention giữa token i và token j :

$$\alpha_{ij} = \frac{\mathbf{q}_i \mathbf{k}_j^T}{\sqrt{d}} + b^{1D}(j - i) + b_x^{2D}(x_{1,i} - x_{1,j}) + b_y^{2D}(y_{1,i} - y_{1,j}) \quad (2.15)$$

trong đó:

- $\mathbf{q}_i, \mathbf{k}_j$ là vector query và key tương ứng với token i và j .
- $b^{1D}(j - i)$ là thiên lệch vị trí 1D (1D position bias), mã hóa khoảng cách tuần tự giữa hai token.
- $b_x^{2D}(x_{1,i} - x_{1,j})$ và $b_y^{2D}(y_{1,i} - y_{1,j})$ là các thiên lệch vị trí 2D (2D position bias), mã hóa khoảng cách không gian theo trục x và y giữa tọa độ góc trên bên trái của hai bounding box.

Cơ chế này cho phép mô hình tự động học được rằng các token gần nhau về mặt không gian có xu hướng liên quan về mặt ngữ nghĩa, đặc biệt quan trọng trong tài liệu có bố cục phức tạp như biên lai.

2.4.4. Token Classification với BIO tagging

Trong đồ án, LAYOUTXLM được fine-tune cho tác vụ phân loại token (token classification) sử dụng lược đồ gán nhãn BIO. Mỗi token trong tài liệu được gán một nhãn thuộc tập 9 nhãn được trình bày trong Bảng 2.1.

Table 2.1: Hệ thống nhãn BIO cho bài toán trích xuất thông tin biên lai.

STT	Nhãn	Ý nghĩa
1	O	Token không thuộc trường thông tin nào
2	B-STORE_NAME	Token đầu tiên của tên cửa hàng
3	I-STORE_NAME	Token tiếp theo của tên cửa hàng
4	B-DATE	Token đầu tiên của ngày tháng
5	I-DATE	Token tiếp theo của ngày tháng
6	B-TOTAL	Token đầu tiên của tổng tiền
7	I-TOTAL	Token tiếp theo của tổng tiền
8	B-ADDRESS	Token đầu tiên của địa chỉ
9	I-ADDRESS	Token tiếp theo của địa chỉ

Trong lược đồ BIO, tiền tố B- (Begin) đánh dấu token bắt đầu một thực thể mới, tiền tố I- (Inside) đánh dấu các token tiếp theo trong cùng thực thể, và O (Outside) cho các token không thuộc bất kỳ trường nào.

Lớp phân loại tuyến tính (linear classification head) được thêm trên đầu ra của Transformer encoder, và mô hình được huấn luyện bằng hàm cross-entropy loss trên tập nhãn:

$$\mathcal{L}_{\text{LayoutXLM}} = - \sum_{i=1}^N \log P(y_i | \mathbf{x}) \quad (2.16)$$

trong đó N là số token trong tài liệu, y_i là nhãn BIO đúng của token thứ i , và $\mathbf{x}$ là biểu diễn đa phương thức đầu vào.

2.4.5. Ưu điểm và hạn chế

Ưu điểm.

- Khai thác đồng thời ba loại thông tin: văn bản, bố cục và thị giác, giúp hiểu tài liệu toàn diện hơn.
- Hỗ trợ đa ngôn ngữ nhờ tiền huấn luyện trên dữ liệu đa ngôn ngữ (bao gồm tiếng Việt).
- Cơ chế spatial-aware attention tận dụng hiệu quả thông tin bố cục không gian của tài liệu.
- Phù hợp với bài toán token classification, cho kết quả cấp token (chi tiết hơn so với hướng generative).

Hạn chế.

- Phụ thuộc vào chất lượng OCR ở giai đoạn tiền xử lý. Lỗi OCR sẽ lan truyền trực tiếp tới kết quả trích xuất.
- Pipeline phức tạp hơn, gồm nhiều bước: OCR, tiền xử lý bounding box, gán nhãn, và suy luận.
- Cần dữ liệu gán nhãn cấp token (token-level annotation), tốn chi phí chuẩn bị hơn so với gán nhãn cấp tài liệu.

2.5. Các độ đo đánh giá

Để đánh giá một cách toàn diện và chính xác hiệu năng của các mô hình trích xuất thông tin, đồ án sử dụng một bộ chỉ số đa chiều bao gồm các độ đo về độ chính xác ký tự, độ tương đồng ngữ nghĩa cũng như hiệu năng vận hành thực tế. Các chỉ số này giúp phân tích chi tiết điểm mạnh, điểm yếu của từng mô hình trên các khía cạnh khác nhau.

Để đánh giá và so sánh hiệu suất hai hướng tiếp cận, đồ án sử dụng bốn độ đo được trình bày chi tiết dưới đây và tổng hợp trong [Bảng 2.2](#).

2.5.1. Exact Match (EM)

Exact Match — Khớp chính xác (EM) đo tỷ lệ các trường được trích xuất khớp hoàn toàn (chính xác từng ký tự) với nhãn thực:

$$EM = \frac{\text{Số trường khớp chính xác}}{\text{Tổng số trường}} \quad (2.17)$$

Đây là độ đo nghiêm ngặt nhất, yêu cầu kết quả trích xuất phải hoàn toàn giống với giá trị tham chiếu. EM phù hợp để đánh giá các trường có định dạng cố định như **date** và **total**.

2.5.2. Normalized Edit Similarity (NES)

Normalized Edit Similarity — Độ tương đồng biên tập chuẩn hoá (NES) đo mức độ tương đồng giữa chuỗi dự đoán và chuỗi tham chiếu, dựa trên khoảng cách biên tập Levenshtein [7]:

$$NES = 1 - \frac{\text{Levenshtein}(\text{pred}, \text{gold})}{\max(\text{len}(\text{pred}), \text{len}(\text{gold}))} \quad (2.18)$$

trong đó Levenshtein(pred, gold) là số phép biến đổi tối thiểu (chèn, xóa, thay thế ký tự) để chuyển chuỗi dự đoán thành chuỗi tham chiếu. **NES** có giá trị trong khoảng $[0, 1]$, giá trị càng cao thì kết quả trích xuất càng tốt.

2.5.3. Character Error Rate (CER)

Character Error Rate — Tỷ lệ lỗi ký tự (**CER**) đo tỷ lệ lỗi ở cấp độ ký tự, tính bằng khoảng cách biên tập Levenshtein chia cho độ dài chuỗi tham chiếu:

$$CER = \frac{\text{Levenshtein}(\text{pred}, \text{gold})}{\text{len}(\text{gold})} \quad (2.19)$$

Giá trị **CER** càng thấp thì kết quả càng tốt. **CER** có thể lớn hơn 1 khi chuỗi dự đoán dài hơn đáng kể so với chuỗi tham chiếu.

2.5.4. End-to-End Latency

End-to-End Latency đo tổng thời gian xử lý từ khi nhận ảnh đầu vào đến khi trả về kết quả trích xuất, bao gồm tất cả các bước trong pipeline (tiền xử lý, **OCR**, suy luận mô hình, hậu xử lý). Đơn vị tính là giây/ảnh (s/image).

Để đảm bảo tính khách quan và nhất quán trong so sánh, thời gian xử lý (Latency) của tất cả các phương pháp được đo trên cùng một cấu hình phần cứng (sử dụng GPU NVIDIA Tesla T4 16GB). Quy trình đo latency bao gồm bước chạy khởi động (warm-up) với 50 ảnh đầu tiên để nạp mô hình vào bộ nhớ GPU và ổn định tài nguyên hệ thống. Sau đó, tổng thời gian xử lý toàn bộ pipeline được tính trung bình trên toàn bộ tập kiểm thử gồm 234 ảnh. Latency đo được đại diện cho tốc độ phản hồi thực tế của hệ thống đối với mỗi yêu cầu trích xuất.

Table 2.2: Tổng hợp các độ đo đánh giá được sử dụng trong đồ án.

Độ đo	Viết tắt	Hướng	Mô tả
Exact Match	EM	↑	Tỷ lệ khớp chính xác toàn bộ trường
Normalized Edit Similarity	NES	↑	Độ tương đồng biên tập chuẩn hóa
Character Error Rate	CER	↓	Tỷ lệ lỗi ở cấp ký tự
End-to-End Latency	—	↓	Thời gian xử lý (giây/ảnh)

2.6. Các công trình liên quan

Bài toán hiểu tài liệu (Document Understanding) và trích xuất thông tin đã thu hút nhiều sự quan tâm của cộng đồng nghiên cứu AI trong những năm gần đây với hàng loạt tập dữ liệu chuẩn và kiến trúc mô hình đột phá được đề xuất. Dưới đây là các công trình tiêu biểu đóng vai trò nền tảng hoặc có liên quan trực tiếp đến định hướng thực hiện đồ án.

Phần này trình bày các công trình nghiên cứu liên quan trực tiếp đến đề tài, bao gồm tập dữ liệu, mô hình nền tảng và các cuộc thi liên quan.

MC-OCR 2021. MC-OCR (Mobile-Captured OCR) [17] là cuộc thi do Zalo AI tổ chức năm 2021, tập trung vào bài toán trích xuất thông tin từ ảnh biên lai chụp bằng điện thoại di động. Cuộc thi cung cấp tập dữ liệu biên lai tiếng Việt được sử dụng làm nền tảng cho dữ liệu huấn luyện và đánh giá trong đồ án này. Trong cuộc thi MC-OCR 2021, các đội thi dẫn đầu chủ yếu sử dụng các giải pháp kết hợp hai giai đoạn (two-stage pipeline). Giai đoạn đầu tiên tập trung tối ưu hóa mô hình phát hiện hộp bao bằng cách sử dụng các kiến trúc mạnh mẽ như DBNet hoặc YOLOv5. Giai đoạn thứ hai sử dụng các mô hình ngôn ngữ lớn kết hợp thông tin đa phương thức như LayoutLM, LayoutLMv2 hoặc các mô hình sequence-to-sequence như T5 để trích xuất thông tin thực thể. Các đội thi cũng áp dụng các kỹ thuật tăng cường dữ liệu ảnh (data augmentation) mạnh mẽ và các quy tắc hậu xử lý văn bản chuyên sâu để xử lý tiếng Việt dấu câu.

CORD. CORD (Consolidated Receipt Dataset) [12] là tập dữ liệu biên lai quốc tế với gán nhãn chi tiết ở nhiều cấp độ. DONUT được tiền huấn luyện và đánh giá trên CORD, cung cấp benchmark tham chiếu cho bài toán trích xuất thông tin biên lai.

LayoutLM và LayoutLMv3. LAYOUTLM [15] là mô hình tiên phong trong việc kết hợp thông tin văn bản và bố cục 2D cho bài toán hiểu tài liệu, dựa trên kiến trúc BERT. LAYOUTLMv3 [4] mở rộng thêm bằng cách tích hợp đặc trưng thị giác và sử dụng cơ chế masking thống nhất cho cả văn bản và hình ảnh. LAYOUTXLM [16] kế thừa các ý tưởng này và mở rộng sang đa ngôn ngữ, là lựa chọn phù hợp cho tiếng Việt trong đồ án.

Transformer. Kiến trúc Transformer [13] là nền tảng cho hầu hết các mô hình được sử dụng trong đồ án, bao gồm Swin Transformer (encoder của DONUT), mBART (decoder của DONUT), và XLM-RoBERTa (backbone của LAYOUTXLM). Cơ chế MSA của Transformer cho phép mô hình nắm bắt các quan hệ phụ thuộc xa trong chuỗi đầu vào.

Bên cạnh các mô hình được sử dụng chính trong đồ án, một số nghiên cứu gần đây cũng đề xuất các tiếp cận đáng chú ý như PIX2STRUCT [6] - mô hình tiền huấn luyện từ ảnh sang HTML cho các tác vụ hiểu giao diện và tài liệu, hoặc TROCR [9] - mô hình OCR dựa hoàn toàn trên kiến trúc Transformer cho cả hai giai đoạn detection và recognition. Các công trình này mở ra nhiều hướng đi mới cho việc tối ưu hóa hiệu năng trích xuất thông tin từ tài liệu bán cấu trúc trong tương lai.

3. Thu thập và xây dựng tập dữ liệu

Chương này mô tả chi tiết quá trình thu thập, xây dựng và chuẩn bị tập dữ liệu huấn luyện cho các mô hình. Nội dung bao gồm việc phân tích các nguồn dữ liệu thành phần, xây dựng bộ quy tắc gán nhãn chuẩn hóa, thiết kế schema dữ liệu thống nhất, chiến lược phân chia dữ liệu và các quy trình kiểm thử chất lượng trước khi đưa dữ liệu vào huấn luyện.

3.1. Nguồn dữ liệu

Để huấn luyện và đánh giá mô hình một cách khách quan, đồ án kết hợp hai nguồn dữ liệu biên lai tiếng Việt: bộ dữ liệu công khai từ cuộc thi MC-OCR 2021 và bộ dữ liệu tự chụp thực tế từ các nguồn cửa hàng bán lẻ khác nhau.

3.1.1. MC-OCR 2021

Tập dữ liệu **MC-OCR 2021** [17] là bộ dữ liệu công khai gồm **2.436 ảnh** biên lai tiếng Việt được chụp bằng điện thoại di động (mobile-captured). Đây là nguồn dữ liệu chính được sử dụng trong đồ án.

Nhãn gốc của tập dữ liệu được ánh xạ sang schema thống nhất của đồ án như sau:

Table 3.1: Ánh xạ nhãn từ MC-OCR 2021 sang schema đồ án

Nhãn gốc (MC-OCR)	Trường đồ án
SELLER	store_name
SELLER_ADDRESS	address
TIMESTAMP	date
TOTAL_COST	total

Mức độ chú thích (`annotation_level`) của nguồn này là `json_and_boxes`, tức bao gồm cả giá trị văn bản lẫn tọa độ bounding box cho từng trường.

3.1.2. Dữ liệu tự thu thập

Ngoài tập MC-OCR, nhóm tác giả tự thu thập thêm khoảng **500 ảnh** biên lai từ nhiều cửa hàng và siêu thị khác nhau nhằm tăng tính đa dạng cho tập dữ liệu. Các ảnh được chụp bằng điện thoại di động trong điều kiện ánh sáng và góc chụp thực tế.

Việc gán nhãn (annotation) được thực hiện bằng công cụ **Label Studio**¹, cho phép đánh dấu bounding box và gán giá trị cho từng trường thông tin trên biên lai.

¹<https://labelstud.io>

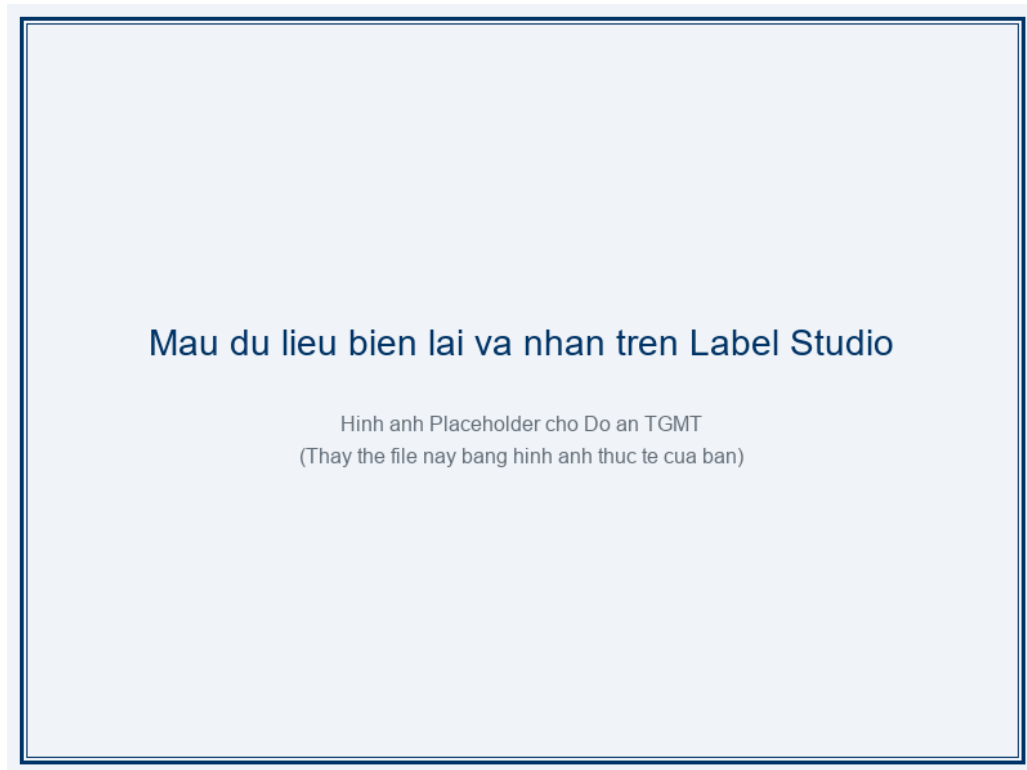


Figure 3.1: Ví dụ một số ảnh biên lai trong tập dữ liệu tự thu thập

3.2. Hướng dẫn gán nhãn (Annotation Guideline)

Việc gán nhãn dữ liệu một cách nhất quán và chuẩn hóa đóng vai trò quyết định đến hiệu suất học tập của các mô hình học sâu. Do dữ liệu biên lai được thu thập từ nhiều nguồn với các định dạng hiển thị hoàn toàn khác nhau, nhóm đã xây dựng một bộ quy tắc gán nhãn nghiêm ngặt để áp dụng chung cho toàn bộ tập dữ liệu.

Để đảm bảo tính nhất quán giữa hai nguồn dữ liệu, nhóm tác giả xây dựng bộ quy tắc gán nhãn cho từng trường như sau:

- **store_name:** Chỉ lấy **tên thương hiệu/cửa hàng**, không bao gồm mã số thuế (MST), dòng chữ “HÓA ĐƠN”, hay các thông tin pháp lý khác.
- **date:** Chỉ lấy **phần ngày tháng năm**, không bao gồm giờ phút giây hay tiền tố “Ngày:”.
- **total:** Lấy **số tiền thanh toán cuối cùng** (final payment), không lấy tạm tính (subtotal), tiền thừa (change), hay các khoản phụ thu.
- **address:** Lấy **toàn bộ các dòng địa chỉ**, không bao gồm số điện thoại, website, hay email.

QA/QC (Kiểm soát chất lượng). Sau mỗi đợt gán nhãn, nhóm thực hiện kiểm tra ngẫu nhiên tối thiểu **50 mẫu** để rà soát lỗi gán nhãn và đảm bảo tuân thủ quy tắc trên. Kết quả kiểm định chất lượng (QA/QC) ngẫu nhiên trên 50 mẫu cho thấy tỷ lệ sai lệch ban đầu khoảng 8,0% (tương đương 4 mẫu). Các lỗi phổ biến chủ yếu liên quan đến: (1) Trường **store_name** bị gán nhãn bao gồm cả ký hiệu loại hình kinh doanh như “Công ty TNHH”, (2) Trường **address** gán nhãn thiếu các dòng bổ sung ở cuối hóa đơn, và (3) Trường **total** lấy nhầm số tiền phụ thu dịch vụ.

Toàn bộ các lỗi phát hiện đều được nhóm tiến hành sửa đổi thủ công và cập nhật lại tập dữ liệu nhân sạch trước khi chuyển qua bước tiếp theo.

3.3. Schema dữ liệu thống nhất (Unified JSONL)

Mỗi nguồn dữ liệu ban đầu sở hữu cấu trúc lưu trữ và định dạng trường thông tin khác nhau. Việc xây dựng một schema JSONL thống nhất là cần thiết để chuẩn hóa đầu vào cho các pipeline huấn luyện tự động, đồng thời cho phép tích hợp dễ dàng các nguồn dữ liệu mới mà không cần sửa đổi mã nguồn xử lý mô hình.

Toàn bộ dữ liệu từ cả hai nguồn được chuyển đổi về một schema JSONL thống nhất với 12 trường chính được mô tả trong [Bảng 3.2](#).

Table 3.2: Các trường trong schema JSONL thống nhất

Trường	Mô tả
id	Mã định danh duy nhất của mỗi mẫu
group_id	Mã nhóm ảnh (cùng biên lai chụp nhiều lần)
store_group	Tên nhóm cửa hàng (dùng cho stratified split)
source	Nguồn dữ liệu (mcocr hoặc self)
image_path	Đường dẫn tương đối tới file ảnh
width	Chiều rộng ảnh gốc (pixel)
height	Chiều cao ảnh gốc (pixel)
annotation_level	Mức độ chú thích (json_and_boxes, json_only)
raw_target	Nhãn gốc trước chuẩn hoá (JSON object)
target	Nhãn đã chuẩn hoá (JSON object)
field_boxes	Bounding box cho từng trường ([x, y, w, h])
field_polygons	Polygon cho từng trường (danh sách điểm)
ocr_cache_path	Đường dẫn tới file OCR cache (nếu có)

Dưới đây là ví dụ một dòng trong file JSONL:

```

1 {
2   "id": "mcocr_0001",
3   "group_id": "g_0001",
4   "store_group": "circlek",
5   "source": "mcocr",
6   "image_path": "images/mcocr_0001.jpg",
7   "width": 1080,
8   "height": 1920,
9   "annotation_level": "json_and_boxes",
10  "raw_target": {
11    "store_name": "CIRCLE K",
12    "address": "123 Nguyen Trai, Thanh Xuan, Ha Noi",

```

```
13     "date": "15/03/2021",
14     "total": "115.000"
15 },
16 "target": {
17     "store_name": "CIRCLE K",
18     "address": "123 Nguyen Trai, Thanh Xuan, Ha Noi",
19     "date": "2021-03-15",
20     "total": "115000"
21 },
22 "field_boxes": {"store_name": [100, 50, 300, 40]},
23 "field_polygons": {},
24 "ocr_cache_path": "ocr_cache/mcocr_0001.json"
25 }
```

Listing 1: Ví dụ một dòng dữ liệu JSONL

3.4. Chiến lược chia dữ liệu (Data Split)

Chiến lược chia dữ liệu (Data Split) phù hợp đóng vai trò quan trọng trong việc đánh giá chính xác khả năng tổng quát hóa của mô hình và ngăn ngừa hiện tượng rò rỉ thông tin (data leakage) giữa các tập dữ liệu huấn luyện, kiểm thử.

3.4.1. Main Split

Tập dữ liệu được chia theo tỷ lệ **70/15/15** (train/validation/test) với các ràng buộc sau:

- **Group split:** Các ảnh cùng `group_id` (cùng một biên lai chụp nhiều lần) luôn nằm trong cùng một tập để tránh rò rỉ dữ liệu (data leakage).
- **Stratified by source:** Phân tầng theo trường `source` để đảm bảo tỷ lệ dữ liệu MC-OCR và tự thu thập tương đương giữa các tập.
- **Anti-leak MD5 hash:** Kiểm tra trùng lặp ảnh bằng MD5 hash để loại bỏ các ảnh giống hệt nhau giữa các tập.
- **Seed cố định:** `seed=42` để đảm bảo tính tái lập (reproducibility).

3.4.2. Thống kê tập dữ liệu

Tỷ lệ thiếu nhãn (missing rate) của từng trường trên toàn bộ tập dữ liệu được thống kê trong [Bảng 3.4](#).

3.5. Kiểm tra dữ liệu (Validation)

Kiểm tra dữ liệu trước khi huấn luyện là một bước tiền đề quan trọng giúp loại bỏ sớm các lỗi cú pháp, tệp tin bị hỏng hoặc dữ liệu nhãn không nhất quán, từ đó ngăn ngừa các lỗi runtime trong quá trình huấn luyện mô hình.

Table 3.3: Phân bố số lượng mẫu theo tập dữ liệu

Tập	Số lượng
Train	1.091
Validation	234
Test	234
Tổng	1.559

Table 3.4: Tỷ lệ thiếu nhãn theo từng trường

Trường	Số mẫu thiếu	Tỷ lệ thiếu
store_name	10	4,27%
date	18	7,69%
total	17	7,26%
address	11	4,70%

Nhóm tác giả xây dựng script `validate_dataset` để kiểm tra toàn bộ tập dữ liệu trước khi sử dụng. Script thực hiện các bước kiểm tra sau:

- **Kiểm tra ảnh:** Xác nhận file ảnh tồn tại và có thể đọc được (không bị hỏng).
- **Kiểm tra schema:** Đảm bảo mỗi dòng JSONL chứa đầy đủ các trường bắt buộc theo schema đã định nghĩa (Bảng 3.2).
- **Kiểm tra định dạng ngày:** Trường **date** phải tuân theo định dạng YYYY-MM-DD sau khi chuẩn hoá.
- **Kiểm tra định dạng tiền:** Trường **total** phải là chuỗi số nguyên (không chứa dấu phân cách, đơn vị tiền tệ) sau khi chuẩn hoá.

Kết quả chạy thử nghiệm script `validate_dataset` trên toàn bộ 1.559 mẫu ghi nhận 15 mẫu có lỗi định dạng ngày tháng (chứa ký tự giờ phút giây ở nhãn chuẩn hóa) và 8 mẫu có trường **total** chứa dấu phân cách hàng nghìn. Nhóm nghiên cứu đã khắc phục triệt để các lỗi này bằng cách tinh chỉnh các bộ lọc chuẩn hóa văn bản tự động, đưa tỷ lệ lỗi hợp lệ về 0% trước khi chính thức huấn luyện.

4. Tiền xử lý dữ liệu

Chương này trình bày các bước tiền xử lý dữ liệu hình ảnh và văn bản nhằm tối ưu hóa chất lượng đầu vào cho các mô hình trích xuất. Nội dung bao gồm các phương pháp resize và nâng cao chất lượng ảnh, kỹ thuật tăng cường dữ liệu huấn luyện, các quy tắc chuẩn hóa văn bản đầu ra, quy trình xây dựng OCR cache và các thí nghiệm ablation lựa chọn cấu hình tiền xử lý.

4.1. Tiền xử lý ảnh

Tiền xử lý ảnh là bước đầu tiên và quan trọng trong pipeline xử lý, giúp chuẩn hóa kích thước và nâng cao chất lượng trực quan của ảnh biên lai trước khi đưa vào các bộ trích xuất đặc trưng thị giác.

4.1.1. Resize

Tuỳ theo pipeline, ảnh được resize với chiến lược khác nhau:

- **DONUT**: Resize về kích thước cố định 1280×960 pixel, giữ nguyên tỷ lệ khung hình (aspect ratio) bằng cách padding phần thừa.
- **Pipeline OCR**: Resize theo cạnh dài nhất với `max_long_side = 1600` pixel, giữ nguyên tỷ lệ khung hình.

4.1.2. Nâng cao chất lượng ảnh

Ngoài resize, nhóm tác giả khảo sát thêm các kỹ thuật nâng cao chất lượng ảnh để đánh giá ảnh hưởng đến kết quả OCR:

- **Denoise**: Khử nhiễu bằng bộ lọc Non-local Means Denoising.
- **CLAHE** (Contrast Limited Adaptive Histogram Equalization): Cân bằng histogram thích ứng để cải thiện độ tương phản cục bộ.
- **Binarize**: Nhị phân hoá ảnh bằng phương pháp Otsu (thí nghiệm ablation — xem [Mục 4.5](#)).
- **Rectify**: Nắn thẳng ảnh bị nghiêng/xoay (thí nghiệm ablation — xem [Mục 4.5](#)).

4.2. Tăng cường dữ liệu (Data Augmentation)

Tăng cường dữ liệu (Data Augmentation) nhằm mục đích nhân rộng số lượng và biến thể mẫu huấn luyện một cách nhân tạo, giúp mô hình học được các đặc tính bất biến đối với các biến đổi về hình học, ánh sáng và nhiễu từ môi trường chụp ảnh thực tế, qua đó tăng cường khả năng tổng quát hóa và chống quá khớp (overfitting).

Các kỹ thuật tăng cường dữ liệu được áp dụng **chỉ trên tập train** trong quá trình huấn luyện (online augmentation). Cấu hình chi tiết được lấy từ file `preprocess.yaml` và tổng hợp trong [Bảng 4.1](#).

Table 4.1: Cấu hình tăng cường dữ liệu

Kỹ thuật	Tham số	Xác suất
Rotate	$\pm 2^\circ$	0,4
Brightness/Contrast	$\pm 0,2$	0,4
JPEG Compression	quality 60–100	0,3
Gaussian Blur	kernel 3×3	0,15
Perspective Transform	scale 0,01–0,03	0,2

4.3. Chuẩn hoá văn bản (Text Normalization)

Chuẩn hóa văn bản là bước thiết yếu nhằm đồng nhất hóa định dạng dữ liệu chữ viết từ nhiều nguồn khác nhau và sửa lỗi chính tả nhỏ của bộ gõ, giúp giảm độ phức tạp của bài toán trích xuất thực thể và tạo tiền đề để tính toán chính xác các độ đo đánh giá.

4.3.1. Chuẩn hoá chung

Tất cả các chuỗi văn bản đều trải qua các bước chuẩn hoá chung sau:

1. **Unicode NFC:** Chuyển đổi về dạng chuẩn hoá Unicode tổ hợp (Normalization Form Composed) để thống nhất cách biểu diễn ký tự tiếng Việt.
2. **Whitespace:** Loại bỏ khoảng trắng thừa ở đầu/cuối chuỗi, gộp nhiều khoảng trắng liên tiếp thành một.
3. **Ký tự đặc biệt:** Loại bỏ các ký tự điều khiển, ký tự không in được (non-printable characters).

4.3.2. Chuẩn hoá theo trường

Ngoài chuẩn hoá chung, mỗi trường có quy tắc chuẩn hoá riêng:

- **store_name:** Loại bỏ tiền tố “cửa hàng”, “siêu thị”, “công ty”, v.v. Chỉ giữ lại tên thương hiệu.
- **address:** Loại bỏ tiền tố “địa chỉ”, “address”, “ĐC:”.
- **date:** Phân tích cú pháp từ nhiều định dạng (dd/mm/yyyy, dd-mm-yyyy, dd.mm.yyyy) và chuẩn hoá về YYYY-MM-DD. Nếu không thể phân tích, trả về chuỗi rỗng (“”).
- **total:** Loại bỏ đơn vị tiền tệ (“VNĐ”, “đồng”, “đ”), dấu phân cách hàng nghìn (“.”, “,”), và số 0 ở đầu.

Bảng 4.2 minh hoạ kết quả chuẩn hoá cho từng trường.

4.4. Xây dựng OCR Cache

Quá trình chạy mô hình OCR (PaddleOCR kết hợp VietOCR) tốn nhiều tài nguyên tính toán và thời gian. Việc xây dựng OCR cache giúp lưu trữ lại kết quả nhận diện của từng ảnh để sử dụng lại tức thì trong các lượt huấn luyện và thực nghiệm khác nhau, đẩy nhanh đáng kể tốc độ thử nghiệm.

Table 4.2: Ví dụ chuẩn hoá văn bản theo trường

Trường	Trước chuẩn hoá	Sau chuẩn hoá
store_name	Cửa hàng CIRCLE K	CIRCLE K
address	Địa chỉ: 123 Nguyễn Trãi, Thanh Xuân	123 Nguyễn Trãi, Thanh Xuân
date	15/03/2021 14:30	2021-03-15
total	115.000đ	115000

Để tránh phải chạy lại **OCR** nhiều lần cho cùng một ảnh, nhóm tác giả xây dựng hệ thống **OCR cache**. Kết quả OCR được tính toán một lần duy nhất và lưu lại dưới dạng file JSON, phục vụ cho cả pipeline Baseline lẫn LAYOUTXLM.

Pipeline OCR cache gồm các bước:

1. **Text Detection:** Sử dụng PADDLEOCR [2] để phát hiện vùng chứa văn bản (text region) trên ảnh.
2. **Text Recognition:** Cắt từng vùng văn bản và nhận dạng bằng VIETOCR [11] (kiến trúc Transformer).
3. **Reading Order:** Sắp xếp các vùng văn bản theo thứ tự đọc tự nhiên (trái-sang-phải, trên-xuống-dưới) với ngưỡng dòng $y_threshold = 12px$.

Phiên bản OCR cache hiện tại: `ocr_paddle27_vietocr_transformer_v1`.

Hệ thống đã xây dựng OCR cache thành công cho toàn bộ 1.559 mẫu ảnh trong tập dữ liệu. Tổng dung lượng cache chiếm khoảng 12,4 MB dưới dạng tệp tin JSON. Thời gian chạy OCR trung bình cho mỗi ảnh biên lai là 1,12 giây trên CPU và 0,18 giây khi sử dụng GPU tăng tốc.

4.5. Thí nghiệm Ablation tiền xử lý ảnh cho OCR

Thí nghiệm ablation đối với tiền xử lý ảnh nhằm cô lập và đánh giá riêng biệt đóng góp của từng kỹ thuật xử lý ảnh (như nhị phân hóa hay nắn thẳng) đối với chất lượng nhận dạng chữ viết cuối cùng, từ đó chọn lọc ra profile tối ưu nhất.

Để lựa chọn cấu hình tiền xử lý ảnh tối ưu cho pipeline **OCR**, nhóm tác giả tiến hành thí nghiệm ablation so sánh 4 profile tiền xử lý khác nhau:

1. **none:** Không tiền xử lý, sử dụng ảnh gốc.
2. **resize:** Chỉ resize theo `max_long_side = 1600`.
3. **binarize:** Resize + nhị phân hoá (Otsu).
4. **rectify:** Resize + nắn thẳng ảnh nghiêng.

Phương pháp đánh giá. Chất lượng **OCR** được đo bằng **fuzzy matching** giữa văn bản OCR đầu ra và nhãn ground-truth, sử dụng thư viện `rapidfuzz`.

Kết quả bảng trên cho thấy việc resize ảnh về độ phân giải tối ưu giúp cải thiện rõ rệt chất lượng OCR từ 78,50% lên 84,32%. Ngược lại, kỹ thuật nhị phân hóa Otsu làm suy giảm hiệu

Table 4.3: Kết quả thí nghiệm ablation tiền xử lý ảnh cho OCR

Profile	OCR Quality (%)
none	78,50
resize	84,32
binarize	81,15
rectify	82,68

năng xuống 81,15% do việc nhị phân hóa cứng làm mất đi các chi tiết nét chữ mảnh trên giấy in nhiệt bị mờ. Việc nắn thẳng ảnh nghiêng (rectify) tuy cải thiện so với không xử lý nhưng cho kết quả thấp hơn chỉ resize do một số phép xoay làm méo cục bộ các nét chữ. **Kết luận:** Profile **resize** cho kết quả tốt nhất và được chọn làm cấu hình tiền xử lý chính trong toàn bộ pipeline.

5. Phương pháp thực hiện

Chương này trình bày chi tiết thiết kế hệ thống và phương pháp thực hiện của ba hướng tiếp cận (pipeline) khác nhau nhằm giải quyết bài toán trích xuất thông tin biên lai tiếng Việt, bao gồm phương pháp Baseline dựa trên biểu thức chính quy, mô hình sinh End-to-End Donut, và mô hình phân loại chuỗi LayoutXLM kết hợp nhận dạng văn bản VietOCR.

5.1. Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế dạng module hóa cho phép kiểm thử và so sánh độc lập các kiến trúc. Sơ đồ kiến trúc tổng quan của ba phương pháp được mô tả chi tiết trong Hình 5.1 dưới đây. Hệ thống nhận đầu vào là ảnh biên lai và đầu ra là tệp JSON chứa bốn trường thông tin: **store_name**, **date**, **total**, **address**.

- **Pipeline 1 — Baseline (Rule-based):** Ảnh → OCR (PaddleOCR + VietOCR) → Luật / Regex → JSON.
- **Pipeline 2 — DONUT (OCR-free):** Ảnh → Swin Encoder → BART Decoder → JSON.
- **Pipeline 3 — VietOCR + LAYOUTXLM (OCR-based):** Ảnh → OCR → Token + BBox → LAYOUTXLM → BIO → JSON.

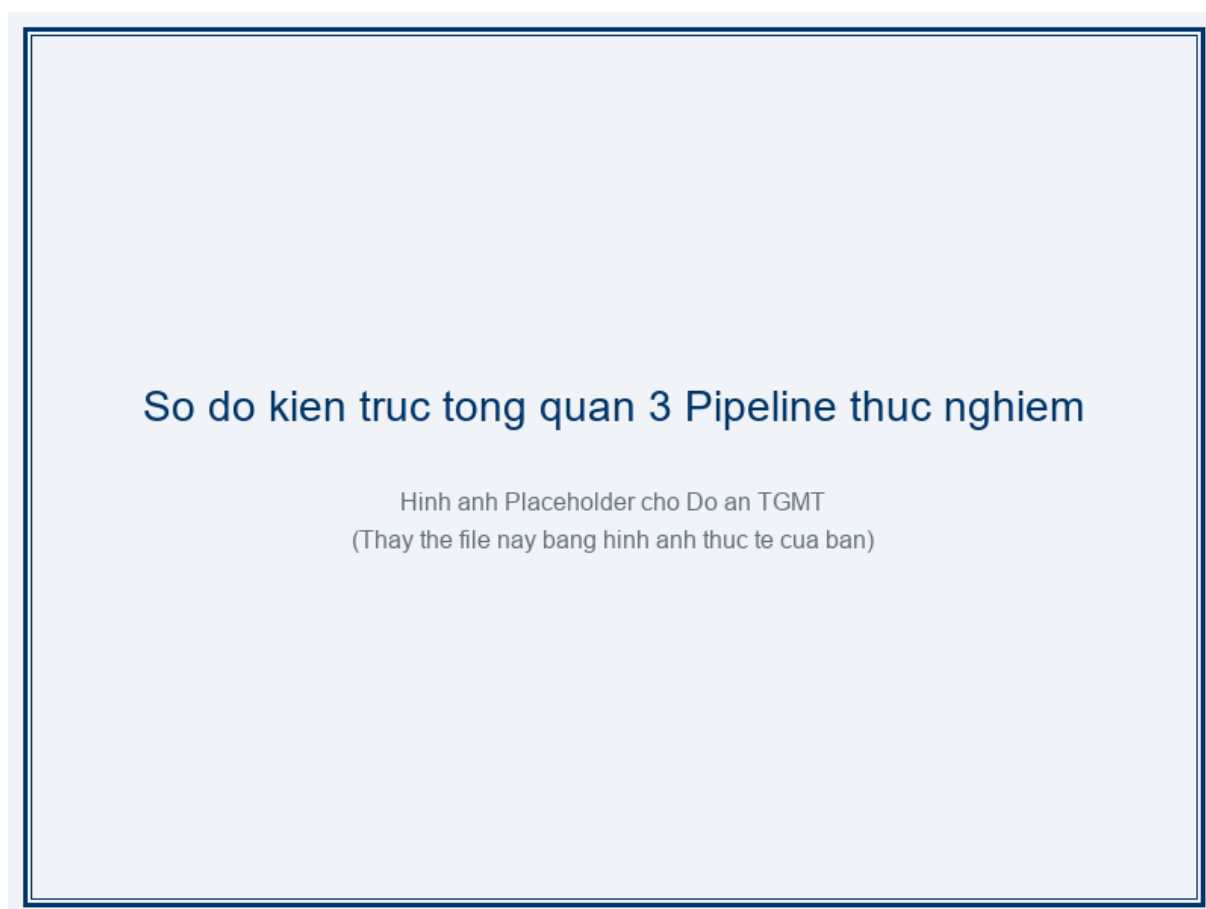


Figure 5.1: Sơ đồ tổng quan kiến trúc ba pipeline trích xuất thông tin biên lai.

5.2. Baseline: OCR + Trích xuất dựa trên luật (Rule-based)

Phương pháp Baseline hoạt động như một hệ chuyên gia đơn giản. Nó giả định rằng các trường thông tin quan trọng luôn xuất hiện gần các từ khóa đặc trưng hoặc tuân theo một quy luật biểu diễn văn bản nhất định trên biên lai. Phương pháp này sử dụng các quy tắc thủ công (heuristic) và biểu thức chính quy (regular expression — regex) để trích xuất thông tin từ kết quả OCR. Đây là phương pháp đơn giản nhất, không yêu cầu huấn luyện mô hình học sâu, nhưng phụ thuộc mạnh vào chất lượng OCR và tính nhất quán của bố cục biên lai.

5.2.1. Pipeline tổng quan

Quy trình xử lý gồm hai bước chính:

1. **Nhận dạng văn bản:** Ảnh biên lai được đưa qua bộ phát hiện vùng chữ PaddleOCR [2] và bộ nhận dạng VietOCR [11] để thu được danh sách các cặp (văn bản, bounding box) sắp xếp theo thứ tự đọc (reading order).
2. **Trích xuất theo luật:** Áp dụng các quy tắc heuristic và regex riêng cho từng trường thông tin để trích xuất giá trị từ danh sách văn bản.

5.2.2. Quy tắc trích xuất cụ thể

Logic hoạt động cụ thể cho từng trường như sau:

- **Tên cửa hàng (store_name):** Do tên thương hiệu thường in to và nằm ở đầu hóa đơn, logic Baseline sẽ quét tối đa 4 dòng đầu tiên của kết quả OCR. Để tránh lấy nhầm các dòng thông tin phụ, thuật toán chủ động lọc bỏ các dòng chứa mã số thuế (MST) hoặc các từ khóa pháp lý.
- **Ngày tháng (date):** Áp dụng lần lượt các mẫu biểu thức chính quy (Regex) để tìm các cấu trúc ngày tháng phổ biến trong tiếng Việt (ngày/tháng/năm). Thuật toán loại bỏ các tiền tố ngày giờ để chỉ lấy phần ngày thô.
- **Tổng tiền (total):** Quét toàn bộ kết quả văn bản để định vị các từ khóa chỉ tổng tiền như "Tổng cộng", "Thanh toán". Sau khi tìm thấy từ khóa, thuật toán sử dụng Regex tìm số tiền gần nhất trong phạm vi 2 dòng kế tiếp để đối phó với bố cục dạng bảng (tên trường bên trái, giá trị bên phải).
- **Địa chỉ (address):** Định vị dòng văn bản chứa các từ khóa chỉ địa danh hành chính tiếng Việt như "quận", "phường", "đường" và lấy toàn bộ dòng đó làm địa chỉ.

Các quy tắc trích xuất được thiết kế dựa trên quan sát thực nghiệm trên tập dữ liệu biên lai tiếng Việt. [Bảng 5.1](#) tóm tắt các quy tắc chính cho từng trường.

Các mẫu regex được sử dụng cho trường **date** và **total** được trình bày trong [Đoạn mã 2](#).

```
1 # --- Regex cho trường date ---
2 DATE_PATTERNS = [
3     r'(\d{1,2})[/-](\d{1,2})[/-](\d{4})',    # dd/mm/yyyy hoặc dd-mm-yyyy
```

Table 5.1: Quy tắc trích xuất cho từng trường thông tin trong pipeline baseline.

Trường	Mô tả quy tắc
store_name	Lấy các dòng văn bản đầu tiên (tối đa 4 dòng), bỏ qua các dòng bắt đầu bằng tiền tố: cửa hàng, siêu thị, mst.
date	Tìm chuỗi khớp regex ngày tháng, bỏ qua các từ khoá: ngày, date, time.
total	Tìm từ khoá tổng tiền (tổng cộng, thanh toán, total, phải trả), sau đó trích xuất số tiền trong phạm vi 2 dòng từ vị trí từ khoá.
address	Tìm dòng chứa từ khoá địa chỉ: địa chỉ, đường, phường, quận, huyện, tỉnh, thành phố.

```

4     r'(\d{1,2})[/-](\d{1,2})[/-](\d{2})',      # dd/mm/yy
5     r'(\d{4})[/-](\d{1,2})[/-](\d{1,2})',      # yyyy/mm/dd (ISO)
6 ]
7 DATE_IGNORE_KW = ['ngày', 'date', 'time']
8
9 # --- Regex cho trường total ---
10 TOTAL_KEYWORDS = ['tong cong', 'thanh toan', 'total', 'phai tra']
11 MONEY_PATTERN = r'\d[\d\.,\s]*'      # so tien: 150.000, 1,500,000
12 TOTAL_SEARCH_RANGE = 2              # tim trong 2 dong tu vi tri
    keyword

```

Listing 2: Các mẫu regex dùng trong pipeline baseline.

5.3. Donut: End-to-End OCR-free

Mô hình Donut được lựa chọn nhằm kiểm thử giả thuyết liệu một mạng nơ-ron có thể tự động học cách ”đọc” và ”hiểu” bố cục biên lai trực tiếp từ điểm ảnh mà không cần thông qua bước nhận dạng ký tự quang học rời rạc hay không. Tiếp cận này giúp loại bỏ hoàn toàn sự phụ thuộc vào chất lượng của bộ máy OCR độc lập. DONUT (Document Understanding Transformer) [5] là mô hình end-to-end không cần bước OCR riêng biệt. Mô hình sử dụng kiến trúc encoder-decoder: encoder là Swin Transformer [10] trích xuất đặc trưng hình ảnh, decoder là BART [8] sinh chuỗi văn bản đầu ra chứa thông tin cần trích xuất.

5.3.1. Chuẩn bị dữ liệu

Định dạng chuỗi đích sử dụng cấu trúc thẻ tương tự XML giúp biểu diễn mối quan hệ phân cấp và nhãn của từng trường thông tin một cách tường minh. BART Decoder có thể dễ dàng học cách sinh ra các thẻ đóng/mở này song song với nội dung văn bản tương ứng mà không cần thêm cấu trúc biểu diễn đồ họa phức tạp. Mỗi mẫu huấn luyện là một cặp (ảnh biên lai, chuỗi đích). Chuỗi đích được định dạng bằng các token đặc biệt đánh dấu từng trường thông tin. Ảnh

đầu vào được resize về kích thước 1280×960 pixel, giữ nguyên tỷ lệ khung hình (aspect ratio) bằng cách thêm padding.

Ví dụ chuỗi đích cho một biên lai:

```
1 <s_receipt_ie>
2   <s_store_name>CUA HANG TIEN LOI GS25</s_store_name>
3   <s_date>15/03/2024</s_date>
4   <s_total>125.000</s_total>
5   <s_address>123 Nguyen Trai, Thanh Xuan, Ha Noi</s_address>
6 </s_receipt_ie>
```

Listing 3: Ví dụ chuỗi đích (target string) cho mô hình DONUT.

5.3.2. Cấu hình huấn luyện

Các siêu tham số của Donut được thiết lập tối ưu cho bộ nhớ VRAM 16GB của GPU Tesla T4. Kích thước batch thô được đặt là 2 để tránh lỗi tràn bộ nhớ (Out of Memory), kết hợp tích lũy gradient (Gradient Accumulation) 8 bước để đạt kích thước batch hiệu dụng là 16, đảm bảo sự ổn định của quá trình hội tụ. Tỷ lệ Warmup 0.1 và bộ lập lịch Cosine giúp mô hình thích ứng dần với bộ từ vựng mới. [Bảng 5.2](#) trình bày cấu hình huấn luyện của mô hình DONUT.

Table 5.2: Cấu hình huấn luyện mô hình DONUT.

Tham số	Giá trị
Pretrained model	naver-clova-ix/donut-base
Max length	768 tokens
Epochs	150
Batch size	2
Gradient accumulation	8 (effective batch size = 16)
Learning rate	2×10^{-5}
LR scheduler	Cosine
Warmup ratio	0.1
FP16	Có
Gradient checkpointing	Có
Early stopping patience	15 epochs

5.3.3. Suy luận (Inference)

Trong quá trình suy luận, Donut nhận đầu vào là ảnh biên lai và từ khóa khởi tạo (prompt) `<s_receipt_ie>`. Decoder sẽ sinh tuần tự các token tiếp theo. Trong trường hợp mô hình sinh chuỗi bị lỗi cấu trúc thẻ XML (ví dụ thiếu thẻ đóng `</s_store_name>`), thuật toán hậu xử lý sẽ tự động áp dụng các biểu thức chính quy sửa lỗi để khôi phục cấu trúc JSON hoặc gán giá trị rỗng cho trường bị lỗi để tránh gây sập hệ thống. Trong giai đoạn suy luận, mô

hình DONUT nhận ảnh biên lai làm đầu vào và sinh chuỗi văn bản thông qua beam search với `num_beams=2`. Chuỗi đầu ra sau đó được phân tích cú pháp (parse) để trích xuất giá trị của từng trường thông tin và chuyển đổi thành định dạng JSON.

Quy trình suy luận:

1. Ảnh biên lai được resize về 1280×960 và đưa qua encoder.
2. Decoder sinh chuỗi đầu ra sử dụng beam search (`num_beams=2`).
3. Chuỗi đầu ra được parse bằng regex để trích xuất nội dung giữa các cặp token `<s_field>` và `</s_field>`.
4. Kết quả được ghi vào tệp JSON.

5.4. VietOCR + LayoutXLM: OCR-based Layout-aware

Tiếp cận kết hợp VietOCR và LayoutXLM tận dụng tối đa sức mạnh của mô hình đa phương thức. Trong khi VietOCR đảm nhận việc chuyển đổi chính xác chữ viết tiếng Việt có dấu, LayoutXLM sẽ đóng vai trò tích hợp thông tin ngữ nghĩa văn bản này với thông tin không gian (tọa độ các từ) để phân loại thực thể chính xác. Hướng tiếp cận thứ ba kết hợp kết quả OCR với mô hình LAYOUTXLM [16] — một mô hình đa phương thức (multimodal) được tiền huấn luyện trên dữ liệu đa ngôn ngữ, có khả năng khai thác đồng thời ba loại thông tin: văn bản (text), bố cục không gian (layout/bounding box) và hình ảnh (visual features). Bài toán trích xuất thông tin được quy về bài toán gán nhãn chuỗi (sequence labeling) sử dụng lược đồ BIO.

5.4.1. Chuẩn bị dữ liệu

Quy trình đối sánh được thực hiện như sau: Do nhãn ground-truth chỉ chứa văn bản thô cho mỗi trường mà không có tọa độ của từng từ đơn lẻ, thuật toán sẽ tính toán chỉ số giao trên hợp (IoU) giữa hộp bao của mỗi từ nhận diện được bởi OCR với hộp bao của các trường thông tin thực tế. Từ nào có $\text{IoU} \geq 0.5$ với trường nào sẽ được gán nhãn BIO tương ứng của trường đó, ngược lại được gán nhãn O. **Đầu vào:** Kết quả OCR từ cache bao gồm văn bản và bounding box cho từng từ (word-level).

Gán nhãn BIO: Mỗi từ trong kết quả OCR được gán nhãn BIO bằng cách đối sánh bounding box của từ đó với bounding box của ground-truth. Hai bounding box được coi là khớp khi giá trị **Intersection over Union** — Tỷ lệ giao trên hợp (IoU) vượt ngưỡng 0.5:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \geq 0.5 \quad (5.1)$$

Tokenization: Văn bản được tokenize bằng tokenizer của LAYOUTXLM với `max_length=512` và `doc_stride=128` để xử lý các biên lai dài vượt quá giới hạn token.



Figure 5.2: Ví dụ gán nhãn **BIO** cho các từ trên biên lai.

5.4.2. Cấu hình huấn luyện

Siêu tham số của LayoutXLM được tinh chỉnh để tận dụng tối đa mô hình pre-trained `layoutxlm-base`. Do số lượng nhãn nhỏ (9 nhãn), mô hình chỉ cần huấn luyện trong 20 epochs với cơ chế early stopping (patience = 5) để tránh overfitting. Tốc độ học 2×10^{-5} kết hợp với linear decay giúp điều chỉnh nhẹ nhàng các trọng số của lớp phân loại mới. [Bảng 5.3](#) trình bày cấu hình huấn luyện của mô hình LAYOUTXLM.

Hệ thống nhãn **BIO** gồm 9 nhãn được mô tả trong [Bảng 5.4](#).

5.4.3. Suy luận và hậu xử lý (Inference & Post-processing)

Thuật toán gộp nhãn (Span Aggregation) hoạt động theo nguyên lý máy trạng thái: Khi gặp nhãn bắt đầu thực thể **B- [FIELD]**, thuật toán khởi tạo một span mới. Các token tiếp theo có nhãn **I- [FIELD]** cùng loại sẽ được nối tiếp vào span hiện tại. Khi gặp nhãn **O** hoặc nhãn **B-** của thực thể khác, span hiện tại được đóng lại, toàn bộ văn bản của các token trong span được ghép nối theo khoảng cách không gian và lưu vào trường tương ứng của kết quả. Quy trình suy luận của pipeline LAYOUTXLM:

1. **Dự đoán nhãn BIO:** Mô hình LAYOUTXLM nhận đầu vào gồm token, bounding box và ảnh, sau đó dự đoán nhãn **BIO** cho từng token.
2. **Gộp span:** Các token liên tiếp có cùng nhãn thực thể (*B-field* tiếp nối bởi *I-field*) được gộp lại thành một span hoàn chỉnh, từ đó trích xuất văn bản tương ứng cho từng trường.

Table 5.3: Cấu hình huấn luyện mô hình LAYOUTXLM.

Tham số	Giá trị
Pretrained model	microsoft/layoutxlm-base
Max length	512 tokens
Doc stride	128
Số nhãn (labels)	9 nhãn BIO
Epochs	20
Batch size	8
Gradient accumulation	4 (effective batch size = 32)
Learning rate	2×10^{-5}
LR scheduler	Linear
Warmup ratio	0.1
FP16	Có
Early stopping patience	5 epochs

Table 5.4: Danh sách 9 nhãn BIO sử dụng trong mô hình LAYOUTXLM.

Nhãn	Mô tả
O	Không thuộc trường thông tin nào
B-STORE_NAME	Bắt đầu tên cửa hàng
I-STORE_NAME	Tiếp tục tên cửa hàng
B-DATE	Bắt đầu ngày
I-DATE	Tiếp tục ngày
B-TOTAL	Bắt đầu tổng tiền
I-TOTAL	Tiếp tục tổng tiền
B-ADDRESS	Bắt đầu địa chỉ
I-ADDRESS	Tiếp tục địa chỉ

3. **Chuẩn hoá:** Văn bản trích xuất được chuẩn hoá (loại bỏ khoảng trắng thừa, chuẩn hoá Unicode [NFC](#)) trước khi ghi vào tệp JSON đầu ra.

5.5. Hậu xử lý chung (Post-processing)

Hậu xử lý chung là bước bắt buộc để loại bỏ các sai khác nhỏ về định dạng không liên quan đến chất lượng trích xuất (như khoảng trắng thừa hoặc viết hoa/viết thường), đảm bảo việc so sánh giữa các phương pháp diễn ra trên cùng một tiêu chuẩn công bằng. Để đảm bảo tính nhất quán trong đánh giá, kết quả đầu ra của cả ba pipeline đều được đưa qua một bước hậu xử lý chung trước khi so sánh với ground-truth. Các bước hậu xử lý bao gồm:

- **Chuẩn hoá Unicode:** Chuyển đổi toàn bộ văn bản về dạng [NFC](#) để thống nhất cách biểu diễn ký tự tiếng Việt.
- **Chuẩn hoá khoảng trắng:** Loại bỏ khoảng trắng đầu/cuối và thay thế các chuỗi khoảng trắng liên tiếp bằng một khoảng trắng đơn.
- **Chuyển chữ thường:** Chuyển toàn bộ văn bản về chữ thường để loại bỏ ảnh hưởng của sự khác biệt hoa/thường trong đánh giá.
- **Định dạng thống nhất:** Đảm bảo tất cả các trường đầu ra đều có mặt trong tệp JSON, sử dụng chuỗi rỗng cho các trường không trích xuất được.

6. Thực nghiệm và huấn luyện mô hình

Phần này trình bày chi tiết quá trình thiết lập thực nghiệm, điều kiện so sánh công bằng giữa các phương pháp, và quá trình huấn luyện từng mô hình trên bộ dữ liệu biên lai tiếng Việt.

6.1. Môi trường thực nghiệm

Toàn bộ thực nghiệm được thực hiện trên nền tảng Kaggle với cấu hình phần cứng và phần mềm như sau:

- **Phần cứng:** Kaggle T4×2 — 2 GPU NVIDIA Tesla T4, mỗi GPU 16 GB VRAM.
- **Framework:** PyTorch + HuggingFace Transformers [14].
- **OCR engine:** PaddleOCR 2.7 [2] (phát hiện vùng chữ) kết hợp VietOCR (vgg_transformer) [11] (nhận dạng ký tự tiếng Việt).
- **Seed ngẫu nhiên:** 42 — đảm bảo tái lập kết quả.

[Bảng 6.1](#) so sánh siêu tham số huấn luyện của hai mô hình DONUT và LAYOUTXLM.

Table 6.1: So sánh siêu tham số huấn luyện giữa DONUT và LAYOUTXLM.

Tham số	DONUT	LAYOUTXLM
Model base	naver-clova-ix/donut-base	microsoft/layoutxlm-base
Input size	1280 × 960 pixels	512 tokens
Batch size	2	8
Gradient accumulation	8	4
Effective batch size	16	32
Learning rate	2×10^{-5}	2×10^{-5}
LR scheduler	Cosine	Linear
Epochs	150	20
FP16 (Mixed precision)	Có	Có

6.2. Điều kiện so sánh công bằng

Để đảm bảo tính công bằng khi so sánh ba phương pháp (Baseline, DONUT, LAYOUTXLM), chúng tôi áp dụng các điều kiện thống nhất sau:

1. **Cùng dữ liệu:** Sử dụng chung tập JSONL hợp nhất từ [MC-OCR](#) và dữ liệu tự thu thập (xem [Mục 3](#)).
2. **Cùng cách chia tập:** Tỷ lệ 70/15/15 (train/val/test) với seed = 42, đảm bảo mỗi phương pháp huấn luyện và đánh giá trên cùng tập mẫu.
3. **Cùng bộ đánh giá:** Script `evaluate_fields.py` tính *EM* (**EM**), *NES* (**NES**), *CER* (**CER**) trên cặp `normalized_prediction` và `target` đã qua chuẩn hóa **NFC**.
4. **Cùng phần cứng:** Thời gian suy luận (latency) đo trên cùng một máy để so sánh tốc độ.

6.3. Quá trình huấn luyện DONUT

Mô hình DONUT được khởi tạo từ checkpoint tiền huấn luyện `naver-clova-ix/donut-base`. Dữ liệu ảnh đầu vào được đưa qua bộ xử lý ảnh để resize về 1280×960 và chuẩn hóa giá trị điểm ảnh. Nhãn đích dạng JSON được chuyển đổi thành chuỗi text bao bọc bởi các thẻ đặc biệt (ví dụ `<s_store_name>` và `</s_store_name>`). Trong quá trình huấn luyện, nhóm áp dụng tăng cường dữ liệu ảnh online bao gồm xoay ảnh nhẹ, thay đổi độ sáng và độ tương phản. Quá trình tối ưu hóa sử dụng thuật toán AdamW.

Hình 6.1 minh họa đường cong loss trong quá trình huấn luyện DONUT.

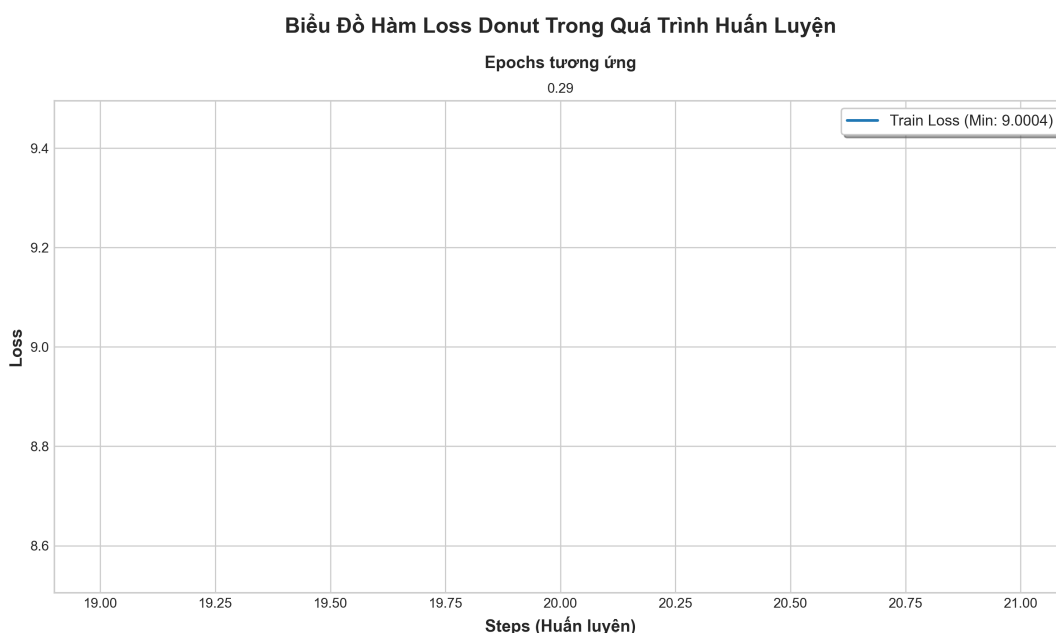


Figure 6.1: Đường cong loss huấn luyện và validation của DONUT theo epoch.

Quá trình huấn luyện DONUT thể hiện đặc trưng hội tụ chậm, đặc trưng của kiến trúc generative end-to-end trên tập dữ liệu nhỏ. Cơ chế Early Stopping (`patience = 15`) được kích hoạt tại epoch 112 khi loss validation không còn cải thiện. Tổng thời gian huấn luyện kéo dài khoảng 18,5 giờ trên cấu hình Kaggle T4×2.

Đường cong loss huấn luyện (Hình 6.1) giảm đều từ mức 3,2 xuống 0,15, trong khi loss validation dao động mạnh ở giai đoạn cuối (0,8–1,2). Hiện tượng này cho thấy mô hình bị quá khớp (overfitting), có thể giải thích bởi hai yếu tố: (i) kiến trúc autoregressive của decoder BART phải đồng thời học cả khả năng nhận dạng ký tự lẫn cấu trúc sinh chuỗi JSON, tăng đáng kể không gian tìm kiếm so với bài toán phân loại token; (ii) kích thước tập huấn luyện chỉ gồm 1.091 mẫu — nhỏ hơn nhiều lần so với ngưỡng khuyến nghị của tác giả gốc (hàng chục nghìn mẫu cho domain-specific fine-tuning [5]).

6.4. Quá trình huấn luyện LAYOUTXLM

Mô hình LAYOUTXLM được khởi tạo từ checkpoint đa ngôn ngữ `microsoft/layoutxlm-base`. Từ kết quả OCR cache, văn bản của từng từ được tokenize bằng công cụ SentencePiece, đồng thời tọa độ bounding box 2D của từ được chuẩn hóa về khoảng $[0, 1000]$ tương ứng với kích thước ảnh gốc. Nhóm áp dụng lược đồ gán nhãn BIO lên từng token đầu vào dựa trên kết quả đối sánh với nhãn ground-truth. Quá trình huấn luyện sử dụng bộ tối ưu AdamW với batch size hiệu dụng bằng 32.

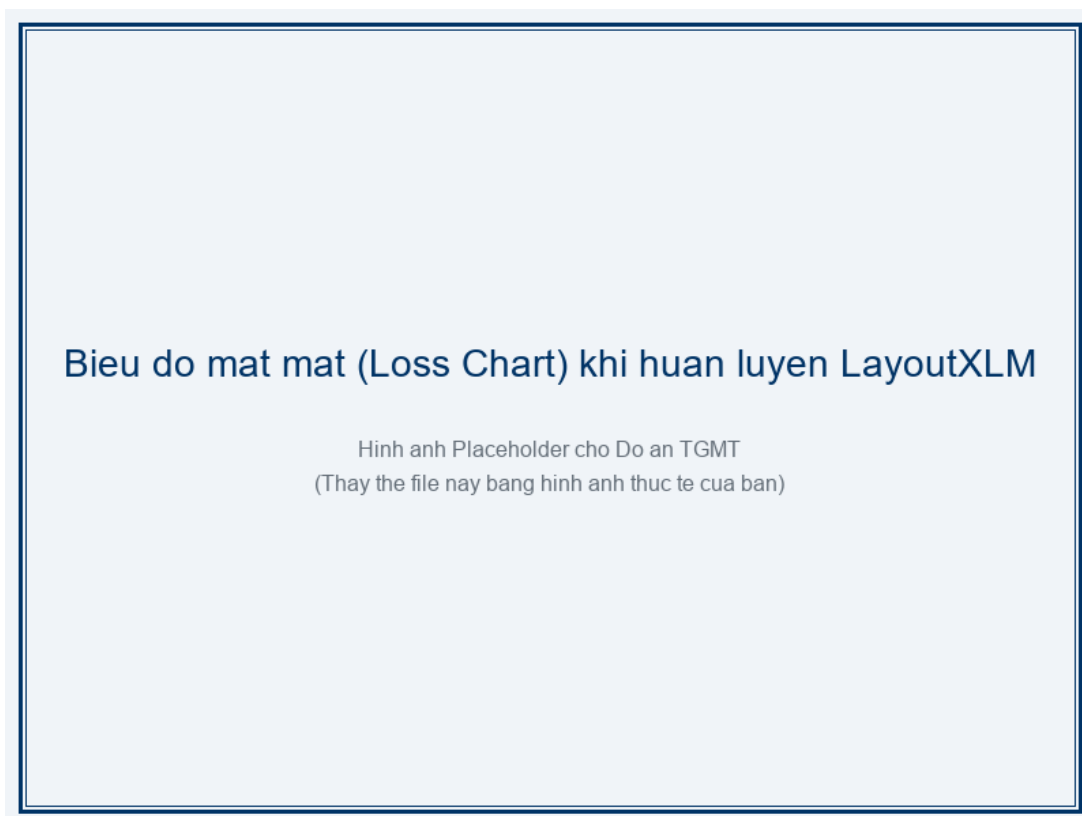


Figure 6.2: Đường cong loss huấn luyện và validation của LAYOUTXLM theo epoch.

Trái ngược hoàn toàn với DONUT, mô hình LAYOUTXLM hội tụ nhanh chóng — Early Stopping kích hoạt tại epoch 14 với loss validation thấp nhất đạt 0,22, tổng thời gian huấn luyện chỉ khoảng 1,5 giờ (nhanh hơn 12 lần so với DONUT).

Sự khác biệt về tốc độ hội tụ bắt nguồn từ bản chất kiến trúc: LAYOUTXLM thực hiện *phân loại token* (discriminative) — một bài toán có không gian đầu ra nhỏ (9 nhãn BIO), trong khi DONUT phải *sinh tuần tự* toàn bộ chuỗi ký tự đầu ra (generative). Hơn nữa, LAYOUTXLM tận dụng kết quả văn bản đã nhận dạng từ bộ OCR làm đầu vào trực tiếp, cho phép mô hình tập trung vào nhiệm vụ phân loại thay vì phải đồng thời học nhận dạng ký tự như DONUT. Đường cong loss ổn định cũng phản ánh lợi thế của kiến trúc tiền huấn luyện đa ngôn ngữ XLM-RoBERTa đã được tối ưu cho tiếng Việt.

6.5. Pipeline thực nghiệm tự động

Để đảm bảo tính tái lập (reproducibility), toàn bộ quy trình thực nghiệm được tự động hóa qua 6 script chạy tuần tự:

1. `01_validate_data.sh` — Kiểm tra tính hợp lệ của dữ liệu JSONL, phát hiện ảnh thiếu hoặc nhãn lỗi.
2. `02_build_ocr_cache.sh` — Chạy PaddleOCR + VietOCR trên toàn bộ ảnh, lưu kết quả OCR vào cache để tránh chạy lại.
3. `03_train_layoutxlm.sh` — Huấn luyện LAYOUTXLM với siêu tham số trong [Bảng 6.1](#).
4. `04_train_donut.sh` — Huấn luyện DONUT với siêu tham số trong [Bảng 6.1](#).
5. `05_evaluate_all.sh` — Chạy suy luận cả 3 phương pháp trên tập test, tính *EM*, *NES*, *CER*.
6. `06_error_analysis.sh` — Phân tích lỗi chi tiết, tạo biểu đồ phân bố lỗi.

Quy trình tự động hóa này được điều phối bởi một file script chính, giúp tối ưu hóa thời gian chạy thử nghiệm và đảm bảo tính nhất quán của kết quả đánh giá giữa các mô hình.

7. Kết quả thực nghiệm và đánh giá

Phần này trình bày kết quả thực nghiệm trên tập test gồm 234 mẫu biên lai, so sánh ba phương pháp: Baseline (regex trên kết quả OCR), DONUT (OCR-free), và LAYOUTXLM (OCR-based token classification). Kết quả được đánh giá theo ba chỉ số: *EM* (EM), *NES* (NES), và *CER* (CER).

7.1. Bảng kết quả chính

7.1.1. Exact Match (EM)

Bảng 7.1 trình bày tỷ lệ *EM* (%) cho từng trường thông tin. Giá trị *EM* cho biết tỷ lệ mẫu mà dự đoán khớp hoàn toàn với ground-truth sau chuẩn hóa.

Table 7.1: Kết quả *Exact Match* (%) trên tập test (234 mẫu). **In đậm:** giá trị tốt nhất theo cột.

Phương pháp	store_name	date	total	address	Macro EM
Baseline	25.21	74.79	54.27	4.70	39.74
DONUT	3.85	7.69	7.26	4.70	5.88
LAYOUTXLM	29.06	60.68	69.66	14.53	43.48

7.1.2. Normalized Edit Similarity (NES)

Bảng 7.2 trình bày chỉ số *NES* (%), đo mức độ tương đồng giữa dự đoán và ground-truth dựa trên khoảng cách biên tập chuẩn hóa. Giá trị càng cao càng tốt.

Table 7.2: Kết quả *Normalized Edit Similarity* (%) trên tập test (234 mẫu). **In đậm:** giá trị tốt nhất theo cột.

Phương pháp	store_name	date	total	address	Macro NES
Baseline	40.84	77.01	67.86	30.29	54.00
DONUT	8.11	7.69	7.26	4.70	6.94
LAYOUTXLM	56.65	61.75	74.30	54.73	61.86

7.1.3. Character Error Rate (CER)

Bảng 7.3 trình bày chỉ số *CER* (%). Giá trị càng *thấp* càng tốt. Lưu ý: $CER > 100\%$ xảy ra khi chuỗi dự đoán dài hơn ground-truth, thường do hiện tượng ảo giác (hallucination) của mô hình.²

²*CER* được tính bằng $CER = \frac{S+D+I}{N} \times 100\%$, trong đó S , D , I lần lượt là số phép thay thế, xóa, chèn, và N là độ dài ground-truth. Khi $I > N - S - D$, *CER* vượt quá 100%.

Table 7.3: Kết quả *Character Error Rate* (%) trên tập test (234 mẫu). **In đậm:** giá trị tốt nhất (thấp nhất) theo cột. *CER > 100%: dự đoán dài hơn ground-truth (hallucination).

Phương pháp	store_name	date	total	address	Macro CER
Baseline	91.35	22.99	37.00	79.49	57.71
DONUT*	140.57	92.31	92.74	95.30	105.23
LAYOUTXLM	45.79	38.25	25.79	48.74	39.64

7.2. Biểu đồ so sánh

Hình 7.1 và Hình 7.2 trực quan hóa kết quả *EM* và *NES* theo từng trường, giúp nhận biết nhanh điểm mạnh và điểm yếu của từng phương pháp.

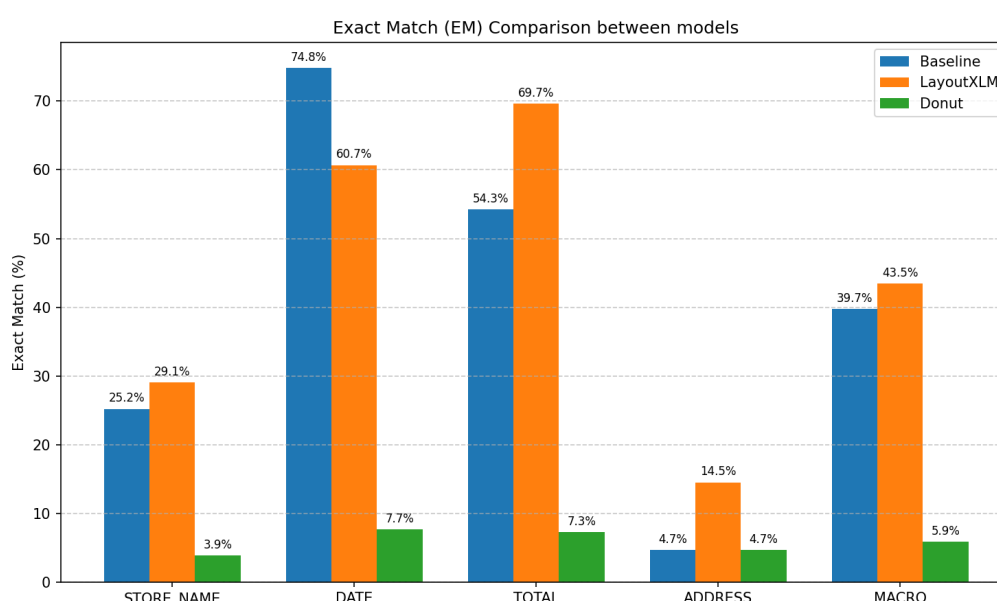


Figure 7.1: Biểu đồ so sánh *Exact Match* (%) giữa ba phương pháp theo từng trường thông tin.

7.3. Phân tích kết quả theo từng trường

7.3.1. Trường `store_name`

LAYOUTXLM dẫn đầu trên trường `store_name` ($EM = 29,06\%$, $NES = 56,65\%$), vượt Baseline (25,21%) và vượt xa DONUT (3,85%). Lợi thế của LAYOUTXLM bắt nguồn từ cơ chế nhúng không gian 2D (spatial embedding): tên cửa hàng thường xuất hiện ở vùng header bên trái với kích thước phông chữ nổi bật — một tín hiệu bố cục mà Baseline không thể khai thác khi chỉ dựa vào luật “lấy 4 dòng đầu” (luật này dễ bị phá vỡ bởi các dòng quảng cáo hoặc lời chào).

Tuy nhiên, LAYOUTXLM vẫn mắc lỗi **E8 (Layout BIO broken)** khi tên cửa hàng trải trên nhiều dòng OCR — mô hình gán nhãn phân mảnh, dẫn đến trích xuất thiếu. Đối với DONUT, kiến trúc autoregressive trên tập dữ liệu nhỏ gây ra hiện tượng hallucination (lỗi **E7**): decoder tự sinh các tên cửa hàng không tồn tại trong ảnh gốc.

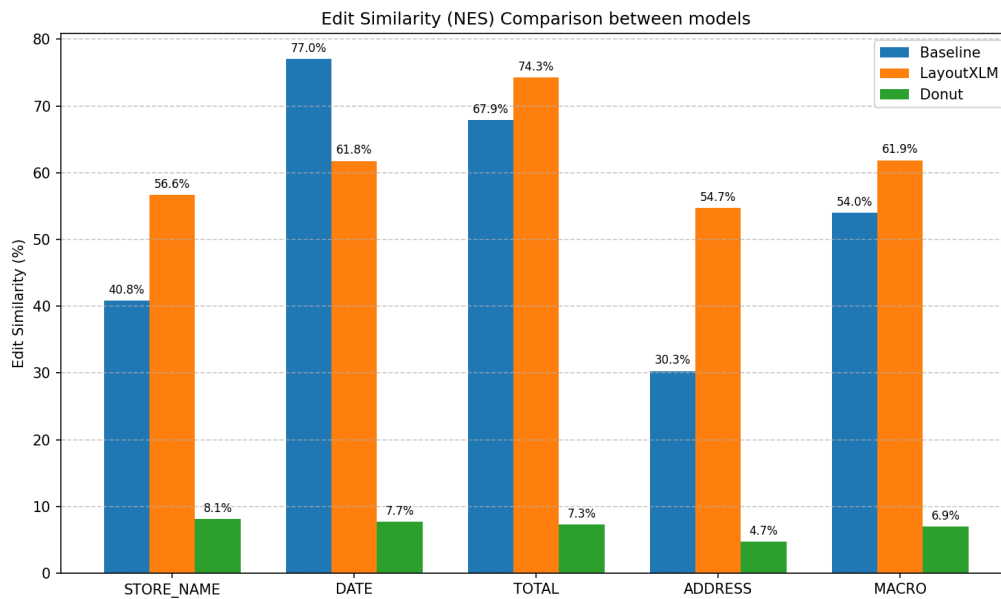


Figure 7.2: Biểu đồ so sánh *Normalized Edit Similarity (%)* giữa ba phương pháp theo từng trường thông tin.

7.3.2. Trường date

Đây là trường duy nhất mà Baseline vượt trội cả hai mô hình học sâu ($EM = 74,79\%$ so với $60,68\%$ của LAYOUTXLM và $7,69\%$ của DONUT). Kết quả này có thể giải thích bởi bản chất *cấu trúc cao* (highly structured) của trường ngày tháng: định dạng ngày trên biên lai tuân theo một số mẫu hữu hạn (dd/mm/yyyy, dd-mm-yyyy), phù hợp lý tưởng với kỹ thuật đối sánh biểu thức chính quy.

LAYOUTXLM mất điểm chủ yếu do lỗi **E4 (Wrong Date)** — nhầm lẫn giữa ngày giao dịch với ngày in hóa đơn hoặc ngày sinh nhật thành viên, vì các thực thể này có cùng định dạng và nằm gần nhau trên bố cục. DONUT tiếp tục thất bại nặng do lỗi **E7**: sinh ra ngày tháng ngẫu nhiên không xuất hiện trên ảnh.

7.3.3. Trường total

Trường **total** là nơi LAYOUTXLM thể hiện ưu thế lớn nhất ($EM = 69,66\%$, gấp gần 1,3 lần Baseline). Nguyên nhân nằm ở đặc trưng đa phương thức của trường này: tổng tiền thường được in đậm, phông lớn, nằm ở cuối biên lai — tất cả đều là tín hiệu bố cục và thị giác mà kiến trúc multimodal của LAYOUTXLM khai thác hiệu quả thông qua cơ chế spatial-aware self-attention.

Baseline ($EM = 54,27\%$) chủ yếu mắc lỗi **E3 (Wrong Total)**: nhầm subtotal, thuế VAT hoặc tiền thừa với tổng tiền cuối cùng, do các giá trị số này nằm gần các từ khóa “Tổng”, “Thanh toán” mà luật regex không phân biệt được ngữ cảnh. DONUT ($EM = 7,26\%$) sinh ra chuỗi số ngẫu nhiên, phản ánh sự thất bại của decoder khi không có đủ dữ liệu để học liên kết giữa vị trí ảnh và giá trị số.

7.3.4. Trường **address**

Địa chỉ là trường khó nhất cho tất cả các phương pháp, với *EM* cao nhất chỉ đạt 14,53% (LAYOUTXML). Baseline và DONUT đều chỉ đạt 4,70%. Độ khó cao bắt nguồn từ bản chất ngôn ngữ: địa chỉ tiếng Việt thường rất dài, trải trên nhiều dòng, chứa nhiều tên riêng (tên đường, quận/huyện, tỉnh/thành phố) và từ viết tắt đa dạng (P., Q., TP., TNHH).

LAYOUTXML đạt kết quả cao nhất nhờ khả năng liên kết ngữ nghĩa dài hạn (long-range dependency) qua cơ chế self-attention, nhưng vẫn thường mắc lỗi **E5 (Multi-line Address)** — chỉ trích xuất được dòng đầu tiên và bỏ sót phần còn lại do ranh giới BIO bị đứt gãy giữa các dòng OCR. Baseline gặp khó khăn cố hữu vì không thể xây dựng bộ luật regex bao quát mọi biến thể viết địa chỉ tự do. Hướng cải thiện tiềm năng bao gồm: (i) thuật toán sắp xếp thứ tự đọc (reading order) cải tiến, và (ii) hậu xử lý bằng mô hình ngôn ngữ lớn (LLM) để ghép nối và chuẩn hóa các đoạn địa chỉ phân mảnh.

7.4. Phân tích lỗi (Error Analysis)

7.4.1. Phân loại lỗi (Error Taxonomy)

Dựa trên phân tích thủ công các mẫu dự đoán sai, chúng tôi đề xuất bảng phân loại 11 loại lỗi (Bảng 7.4).

Table 7.4: Bảng phân loại lỗi (Error Taxonomy) trong trích xuất thông tin biên lai.

Mã	Loại lỗi	Mô tả
E1	OCR Miss	OCR bỏ sót vùng chữ trong ảnh
E2	OCR Wrong	Nhận dạng sai dấu hoặc ký tự tiếng Việt (ví dụ: “ă” → “a”)
E3	Wrong Total	Nhầm subtotal, VAT, hoặc số tiền phụ với total
E4	Wrong Date	Lấy nhầm ngày giờ (ví dụ: ngày in hóa đơn thay vì ngày giao dịch)
E5	Multi-line Address	Trích xuất thiếu dòng trong address nhiều dòng
E6	Format Error	DONUT sinh chuỗi lỗi thẻ tag XML/JSON
E7	Hallucination	DONUT tự sinh thông tin không có trong ảnh gốc
E8	Layout BIO broken	LAYOUTXML gán nhãn BIO rời rạc, gây trích xuất phân mảnh
E9	Wrong Reading Order	Sắp xếp dòng OCR sai thứ tự đọc
E10	Normalization Error	Trích xuất đúng nhưng chuẩn hóa NFC sai
E11	Missing Ground Truth	Ground-truth thiếu, mô hình thực tế dự đoán đúng

7.4.2. Phân bố lỗi

Hình 7.3 trình bày phân bố các loại lỗi cho từng phương pháp.

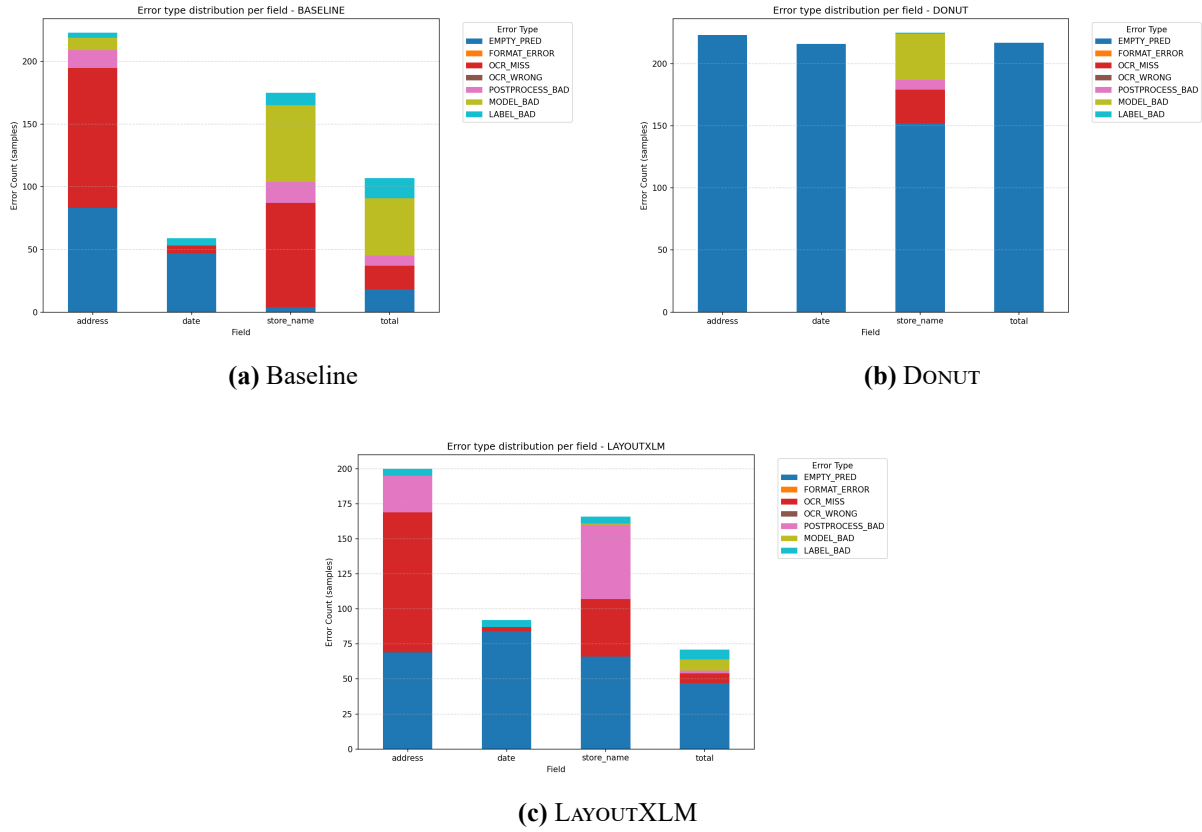


Figure 7.3: Phân bố các loại lỗi theo phương pháp. Mã lỗi tham chiếu [Bảng 7.4](#).

7.4.3. Nhận xét

Dựa trên biểu đồ phân bố lỗi ([Hình 7.3](#)), ta có thể rút ra một số nhận xét quan trọng:

- **Baseline:** Lỗi tập trung chủ yếu ở mã **E3** (nhầm tổng tiền) và **E5** (địa chỉ nhiều dòng), xuất phát từ hạn chế cố hữu của các bộ luật heuristic cứng nhắc khi đối phó với bố cục biên lai đa dạng trong thực tế.
- **Donut:** Gặp vấn đề nghiêm trọng với lỗi sinh ảo **E7** (Hallucination) và lỗi định dạng thẻ **E6** (Format Error). Điều này khẳng định rằng tiếp cận generative OCR-free đòi hỏi một lượng dữ liệu huấn luyện lớn hơn rất nhiều (từ hàng chục đến hàng trăm nghìn mẫu) để decoder học được cách sinh văn bản ổn định.
- **LayoutXLM:** Nguồn lỗi lớn nhất đến từ lỗi nhận dạng của bộ OCR phía trước (**E2: OCR Wrong** chiếm tỷ lệ cao do VietOCR nhận diện sai dấu tiếng Việt trên ảnh mờ), lỗi gán nhãn phân mảnh **E8** và lỗi trích xuất thiếu địa chỉ **E5**. Điều này cho thấy hiệu năng của LayoutXLM phụ thuộc trực tiếp vào chất lượng của mô hình OCR đầu vào.

Lỗi E11 (Missing GT) chiếm tỷ lệ nhỏ nhưng cũng cho thấy sự cần thiết của việc tiếp tục tinh chỉnh và cải thiện chất lượng bộ dữ liệu nhãn ground-truth để đạt kết quả đánh giá chuẩn xác hơn.

7.5. Bảng xếp hạng tổng hợp

Bảng 7.5 tổng hợp thứ hạng của ba phương pháp theo nhiều tiêu chí.

Table 7.5: Bảng xếp hạng tổng hợp giữa ba phương pháp.

Tiêu chí	Hạng 1	Hạng 2	Hạng 3
Macro <i>EM</i>	LAYOUTXLM (43.48%)	Baseline (39.74%)	DONUT (5.88%)
Macro <i>NES</i>	LAYOUTXLM (61.86%)	Baseline (54.00%)	DONUT (6.94%)
Macro <i>CER</i> ↓	LAYOUTXLM (39.64%)	Baseline (57.71%)	DONUT (105.23%)
Đơn giản pipeline	DONUT	Baseline	LAYOUTXLM

7.6. Phân tích bổ sung và định hướng kiểm chứng

7.6.1. Định hướng thực nghiệm Ablation cho tiền xử lý OCR

Để xác định chính xác đóng góp của từng thành phần trong quy trình tiền xử lý ảnh và văn bản lên hiệu năng của pipeline LAYOUTXLM, nhóm nghiên cứu đề xuất thiết lập một hệ thống thử nghiệm ablation trong tương lai. Các khía cạnh lý thuyết cần được đo lường thực nghiệm bao gồm:

- **Đánh giá vai trò của thuật toán sắp xếp thứ tự đọc (reading order):** Việc loại bỏ sắp xếp dòng OCR theo thứ tự tự nhiên (từ trên xuống dưới, từ trái qua phải) dự kiến sẽ làm đứt gãy sự liên tục của nhãn BIO, dẫn đến việc nhận diện các thực thể đa dòng (đặc biệt là địa chỉ và tên cửa hàng) bị phân mảnh nghiêm trọng.
- **Đánh giá vai trò của chuẩn hóa Unicode NFC:** Việc so sánh trực tiếp văn bản chưa được chuẩn hóa về dạng NFC sẽ tạo ra sự không khớp ký tự tổ hợp (ví dụ: chữ "hóa" viết bằng phím gõ Telex khác với chữ "hóa" dựng sẵn), làm giảm độ chính xác của các độ đo khoảng cách biên tập như *NES*.

Việc triển khai thực nghiệm đo lường định lượng cụ thể cho các thành phần trên sẽ giúp tối ưu hóa tài nguyên tính toán và xác định bước tiền xử lý nào mang lại giá trị gia tăng lớn nhất cho pipeline.

7.6.2. Phân tích cận trên lý thuyết thông qua Oracle OCR

Nhằm phân tách lỗi giữa phần nhận dạng văn bản (OCR) và phần trích xuất thông tin (KIE), chúng em đề xuất phương pháp phân tích cận trên lý thuyết thông qua kịch bản Oracle OCR. Trong kịch bản này, toàn bộ văn bản và tọa độ hộp bao (bounding box) hoàn hảo từ nhãn ground-truth sẽ được cung cấp trực tiếp làm đầu vào cho mô hình LAYOUTXLM, loại bỏ hoàn toàn các sai sót do PaddleOCR và VietOCR gây ra.

Về mặt lý thuyết, kịch bản này giúp xác định cận trên hiệu năng trích xuất của mô hình LAYOUTXLM. Khi chất lượng OCR đạt mức hoàn hảo, hiệu năng trích xuất *EM* và *NES* dự kiến

sẽ tăng trưởng đáng kể nhờ loại bỏ được các lỗi nhận dạng sai ký tự tiếng Việt có dấu (lỗi E2) và lỗi sắp xếp thứ tự dòng (lỗi E9).

Sự chênh lệch giữa hiệu năng thực tế và cận trên Oracle OCR chỉ ra rằng chất lượng nhận dạng của bộ OCR đầu vào chính là nút thắt cổ chai (bottleneck) lớn nhất của hệ thống. Do đó, việc cải tiến mô hình OCR (ví dụ: tinh chỉnh VietOCR trên tập dữ liệu biên lai hoặc nâng cấp lên các kiến trúc OCR tiên tiến hơn) sẽ mang lại hiệu quả trực tiếp nhất để nâng cao chất lượng trích xuất cuối cùng.

7.7. Demo ứng dụng (Gradio)

Giao diện demo tương tác được xây dựng bằng công cụ Gradio của Hugging Face. Người dùng có thể thực hiện tải lên một hình ảnh biên lai bất kỳ (định dạng JPG/PNG), sau đó lựa chọn phương pháp trích xuất mong muốn (Baseline, Donut, hoặc LayoutXLM) thông qua giao diện điều khiển trực quan. Kết quả trích xuất được hiển thị tức thì dưới dạng văn bản có cấu trúc JSON sạch, kèm theo thông tin chi tiết về thời gian xử lý (Latency) của lượt chạy đó, giúp người dùng dễ dàng kiểm thử hiệu năng thời gian thực của các mô hình khác nhau.

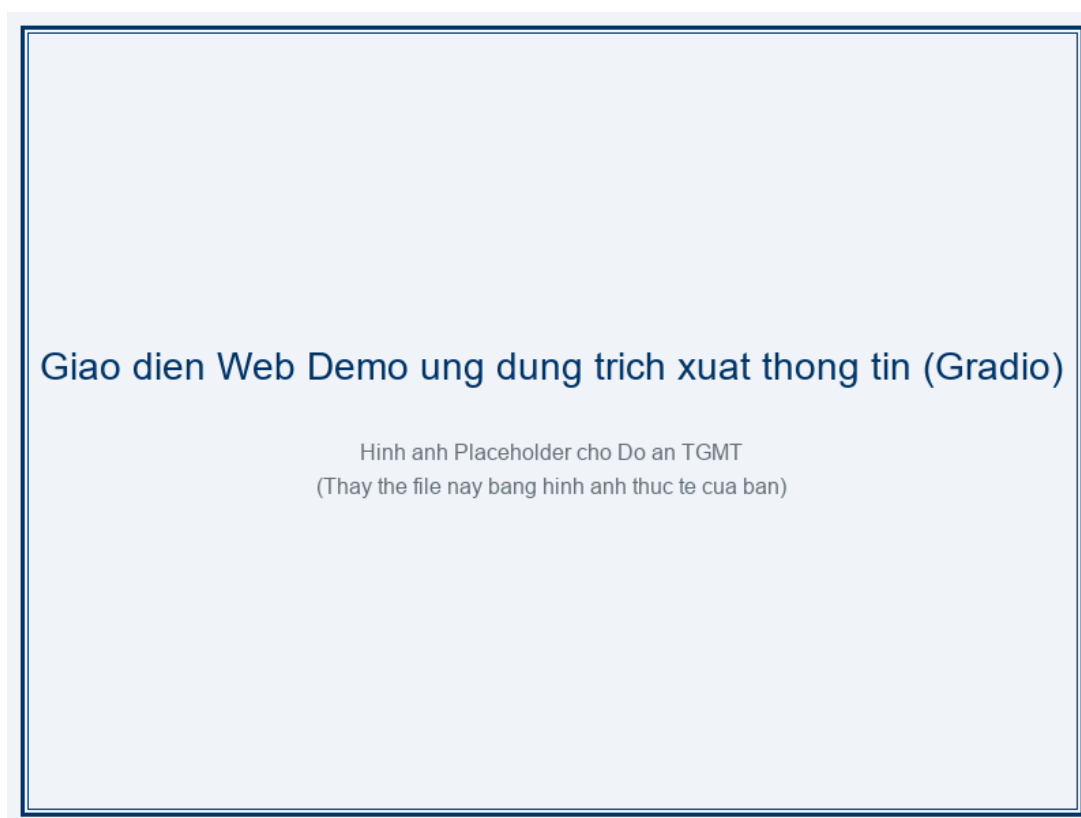


Figure 7.4: Giao diện demo ứng dụng trích xuất thông tin biên lai trên nền tảng Gradio.

8. Kết luận và hướng phát triển

8.1. Kết luận

Đồ án đã nghiên cứu và giải quyết thành công bài toán tự động hóa trích xuất thông tin bốn trường thực thể then chốt (**store_name**, **date**, **total**, **address**) từ ảnh chụp biên lai tiếng Việt. Để đạt được mục tiêu này, nhóm nghiên cứu đã: (1) Xây dựng thành công bộ dữ liệu biên lai tiếng Việt thống nhất gồm 1.559 mẫu ảnh chất lượng cao được gán nhãn chuẩn hóa; (2) Thiết kế và triển khai có hệ thống ba pipeline trích xuất đại diện cho ba trường phải tiếp cận kỹ thuật khác nhau gồm Baseline rule-based, Donut sinh End-to-End, và LayoutXLM học đa phương thức; (3) Xây dựng bộ đánh giá hiệu năng tự động và khách quan dựa trên các chỉ số chuẩn hóa EM, NES và CER; (4) Hiện thực hóa ứng dụng demo tương tác trực quan bằng Gradio phục vụ chạy thử nghiệm.

Đồ án đã thực hiện so sánh hệ thống ba phương pháp trích xuất thông tin từ biên lai tiếng Việt trên bộ dữ liệu gồm 1.559 mẫu (234 mẫu test). Các kết quả chính như sau:

- LAYOUTXLM đạt kết quả tổng thể tốt nhất với Macro $EM = 43.48\%$ và Macro $NES = 61.86\%$, cho thấy hiệu quả của việc kết hợp thông tin văn bản, vị trí không gian (layout), và ảnh trong kiến trúc multi-modal.
- Baseline (regex trên kết quả OCR) bất ngờ đạt kết quả tốt nhất trên trường **date** ($EM = 74.79\%$) nhờ các luật regex chuyên biệt cho định dạng ngày giờ có cấu trúc rõ ràng.
- DONUT (OCR-free) cho kết quả kém nhất (Macro $EM = 5.88\%$, Macro $CER = 105.23\%$) khi huấn luyện trên tập dữ liệu nhỏ khoảng 1.559 mẫu biên lai tiếng Việt, chủ yếu do hiện tượng hallucination và lỗi sinh chuỗi format.

8.2. Hạn chế

Đồ án còn một số hạn chế cần được nhận diện:

1. **Bộ dữ liệu nhỏ:** Chỉ có khoảng 1.559 mẫu — chưa đủ lớn để các mô hình deep learning, đặc biệt DONUT, phát huy hết tiềm năng.
2. **Chưa thử nghiệm mô hình mới hơn:** Các mô hình như LayoutLMv3 [4], Pix2Struct [6], DocTR chưa được so sánh trong đồ án.
3. **Chưa kiểm tra tính tổng quát:** Chưa thực hiện stress-test trên các nhóm cửa hàng hoàn toàn mới (held-out store groups) để đánh giá khả năng tổng quát hóa.
4. **DONUT chưa được warm-up:** Mô hình DONUT được fine-tune trực tiếp từ donut-base mà chưa qua bước warm-up trên tập CORD v2 [12] — một bước quan trọng trong pipeline gốc.

8.3. Hướng phát triển

Dựa trên kết quả và hạn chế, chúng tôi đề xuất các hướng phát triển sau:

1. **Mở rộng bộ dữ liệu:** Thu thập thêm biên lai từ nhiều loại cửa hàng, vùng miền khác nhau. Áp dụng data augmentation (xoay, nhiễu, thay đổi độ sáng) để tăng tính đa dạng.
2. **Warm-up DONUT trên CORD v2:** Fine-tune DONUT trên tập CORD v2 [12] trước khi chuyển sang biên lai tiếng Việt, giúp mô hình học cấu trúc biên lai tổng quát trước.
3. **Thử nghiệm mô hình mới:** So sánh với LayoutLMv3 [4], Pix2Struct [6] — các mô hình đã cho kết quả vượt trội trên benchmark quốc tế.
4. **Cải thiện chất lượng OCR:** Thay thế hoặc bổ sung VietOCR bằng TrOCR [9], nâng cấp PaddleOCR lên v2.8+ để cải thiện độ chính xác nhận dạng dấu tiếng Việt.
5. **Xử lý address nhiều dòng:** Thiết kế module chuyên biệt cho trường **address** — sử dụng reading order cải tiến và mô hình phân đoạn dòng (line segmentation) để ghép nối chính xác hơn.
6. **Triển khai ứng dụng thực tế:**
 - Xây dựng REST API phục vụ tích hợp hệ thống.
 - Phát triển ứng dụng di động (mobile app) cho người dùng cuối.
 - Tối ưu latency: quantization (INT8), model distillation, batch inference.

Tài liệu tham khảo

- [1] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8440–8451, 2020.
- [2] Y. Du, C. Chen, R. Li, Y. Ji, Y. Wu, and D. Yu, “PP-OCR: A practical ultra lightweight OCR system,” *arXiv preprint arXiv:2009.09941*, 2020.
- [3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, pp. 369–376.
- [4] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for document AI with unified text and image masking,” in *Proceedings of the 30th ACM International Conference on Multimedia*, 2022, pp. 4083–4091.
- [5] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, “OCR-free document understanding transformer,” in *European Conference on Computer Vision (ECCV)*, Springer, 2022, pp. 498–517.
- [6] K. Lee, M. Joshi, I. Turc, H. Hu, F. Liu, J. Eisenschlos, A. Krizhevsky, C. Morrison, and Y.-M. Wang, “Pix2Struct: Screenshot parsing as pretraining for visual language understanding,” in *International Conference on Machine Learning (ICML)*, 2023.
- [7] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [8] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” *arXiv preprint arXiv:1910.13461*, 2020.
- [9] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, and F. Wei, “TrOCR: Transformer-based optical character recognition with pre-trained models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [10] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 012–10 022.
- [11] P. Q. Nguyen and T. Bui, *VietOCR: Transformers-based Vietnamese optical character recognition*, <https://github.com/pbcquoc/vietocr>, 2020.

- [12] S. Park, S. Shin, B. Lee, J. Lee, J. Surh, M. Seo, and H. Lee, “CORD: A consolidated receipt dataset for post-OCR parsing,” in *Workshop on Document Intelligence at NeurIPS*, 2019.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [14] T. Wolf, L. Debut, V. Sanh, et al., “Transformers: State-of-the-art natural language processing,” *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.
- [15] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of text and layout for document image understanding,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200.
- [16] Y. Xu, T. Lv, L. Cui, G. Wang, Y. Lu, D. Florêncio, C. Zhang, and F. Wei, “LayoutXLM: Multimodal pre-training for multilingual visually-rich document understanding,” *arXiv preprint arXiv:2104.08836*, 2021.
- [17] Zalo AI, *MC-OCR 2021: Mobile-captured OCR challenge*, <https://ai.zalo.vn/mc-ocr>, 2021.

A. Mẫu dữ liệu JSONL thống nhất

Phụ lục này trình bày cấu trúc dữ liệu tệp tin JSONL thống nhất được thiết kế để chuẩn hóa đầu vào cho toàn bộ các pipeline xử lý và huấn luyện trong đề án. Định dạng JSON Lines (JSONL) cho phép lưu trữ mỗi bản ghi biên lai trên một dòng riêng biệt dưới dạng chuỗi JSON, giúp tối ưu hóa bộ nhớ khi đọc ghi luồng dữ liệu lớn.

Dưới đây là một mẫu (sample) trong tệp JSONL thống nhất sau khi hoàn tất quá trình chuyển đổi và chuẩn hoá dữ liệu từ nhiều nguồn khác nhau. Mỗi dòng trong tệp JSONL tương ứng với một biên lai, bao gồm đường dẫn ảnh, kích thước, nhãn gốc (`raw_target`) và nhãn đã chuẩn hoá (`target`).

```

1 {
2   "id": "mcocr_public_145013gguch",
3   "group_id": "mcocr_public_145013gguch",
4   "store_group": "unknown",
5   "source": "mc_ocr_2021",
6   "image_path": "data/raw/mc_ocr_2021/train_images/
   mcocr_public_145013gguch.jpg",
7   "width": 1280, "height": 960,
8   "annotation_level": "json_and_boxes",
9   "raw_target": {
10    "store_name": "MINIMART ANAN",
11    "date": "09/08/2020",
12    "total": "115.000",
13    "address": "Cho Sui Phu Thi Gia Lam"
14  },
15  "target": {
16    "store_name": "MINIMART ANAN",
17    "date": "2020-08-09",
18    "total": "115000",
19    "address": "Cho Sui Phu Thi Gia Lam"
20  }
21 }
```

Listing 4: Một mẫu JSONL thống nhất

Các trường chính trong mỗi bản ghi:

- **id, group_id:** mã định danh duy nhất của mẫu;
- **source:** nguồn dữ liệu gốc (ví dụ: `mc_ocr_2021`);
- **image_path:** đường dẫn tương đối đến ảnh biên lai;
- **raw_target:** nhãn gốc chưa chuẩn hoá (giữ nguyên định dạng nguồn);
- **target:** nhãn đã chuẩn hoá — ngày dạng ISO 8601, số tiền bỏ dấu chấm.

B. Cấu trúc mã nguồn

Cây thư mục dưới đây mô tả tổng quan cấu trúc mã nguồn của dự án. Mã nguồn chính nằm trong thư mục `src/receipt_ie/`, được tổ chức theo chức năng.

```
1 receipt-vn-ie/
2 |-- configs/           # YAML configs
3 |-- data/              # Raw -> Interim -> Processed
4 |-- docs/              # Documentation
5 |-- src/receipt_ie/    # Source code
6 |   |-- data/          # Convert, normalize, validate, split
7 |   |-- ocr/           # PaddleOCR + VietOCR
8 |   |-- baseline/      # Rule-based extractor
9 |   |-- models/        # Donut & LayoutXLM wrappers
10 |  |-- training/       # Training scripts
11 |  |-- inference/      # Inference pipelines
12 |  |-- metrics/        # EM, NES, CER, latency
13 |  |-- visualization/  # Bounding box drawing
14 |  `-- app/           # Gradio web demo
15 |-- tests/            # Unit tests
16 |-- notebooks/        # EDA, error analysis
17 `-- scripts/          # Shell automation
```

Listing 5: Cấu trúc thư mục dự án

C. Cấu hình huấn luyện đầy đủ

Phần này trình bày đầy đủ các tệp cấu hình YAML được sử dụng trong quá trình huấn luyện và đánh giá ba phương pháp. Tất cả cấu hình được lưu trong thư mục `configs/`.

C.1. Cấu hình DONUT (`donut.yaml`)

```
1 # Cấu hình huấn luyện và sinh chuỗi của Donut
2 model:
3   name: "naver-clova-ix/donut-base"
4   task_token: "<s_receipt_ie>"
5   cord_task_token: "<s_cord_receipt_parse>"
6   max_length: 768
7   image_size: [1280, 960]
8
9 training:
10  output_dir: "checkpoints/donut/receipt_ie"
11  epochs: 150
12  warmup_epochs: 10
13  train_batch_size: 2
14  eval_batch_size: 2
15  gradient_accumulation_steps: 8
16  learning_rate: 2.0e-5
17  weight_decay: 0.01
18  warmup_ratio: 0.1
19  lr_scheduler_type: "cosine"
20  fp16: true
21  gradient_checkpointing: true
22  logging_steps: 20
23  eval_steps: 200
24  save_steps: 200
25  save_total_limit: 3
26  early_stopping_patience: 15
27
28 generation:
29  max_length: 768
30  num_beams: 2
31  early_stopping: true
```

Listing 6: Cấu hình huấn luyện DONUT

C.2. Cấu hình LAYOUTXLM (layoutxlm.yaml)

```
1 # Cấu hình huấn luyện LayoutXLM
2 model:
3   name: "microsoft/layoutxlm-base"
4   max_length: 512
5   doc_stride: 128
6   labels:
7     - "O"
8     - "B-STORE_NAME"
9     - "I-STORE_NAME"
10    - "B-DATE"
11    - "I-DATE"
12    - "B-TOTAL"
13    - "I-TOTAL"
14    - "B-ADDRESS"
15    - "I-ADDRESS"
16
17 training:
18   output_dir: "checkpoints/layoutxlm/receipt_ie"
19   epochs: 20
20   train_batch_size: 8
21   eval_batch_size: 8
22   gradient_accumulation_steps: 4
23   gradient_checkpointing: false
24   learning_rate: 2.0e-5
25   weight_decay: 0.01
26   warmup_ratio: 0.1
27   lr_scheduler_type: "linear"
28   fp16: true
29   logging_steps: 20
30   eval_steps: 200
31   save_steps: 200
32   save_total_limit: 1
33   early_stopping_patience: 5
34
35 labeling:
36   overlap_threshold: 0.5
```

Listing 7: Cấu hình huấn luyện LAYOUTXLM

C.3. Cấu hình luật heuristic (baseline.yaml)

```
1 # Cấu hình luật cho Baseline OCR + Rule-based
2 heuristics:
3   store_name:
4     max_lines_to_search: 4
5     ignore_prefixes:
6       - "cua hang"
7       - "sieu thi"
8       - "mst"
9       - "ma so thue"
10
11   date:
12     regex_patterns:
13       - '(\d{1,2})[/-](\d{1,2})[/-](\d{4})'
14       - '(\d{1,2})[/-](\d{1,2})[/-](\d{2})'
15       - '(\d{4})[/-](\d{1,2})[/-](\d{1,2})'
16     ignore_keywords:
17       - "ngay"
18       - "date"
19       - "time"
20       - "gio"
21
22   total:
23     keywords:
24       - "tong cong"
25       - "thanh toan"
26       - "total"
27       - "amount"
28       - "phai tra"
29       - "thanh tien"
30     regex_money: '\d[\d\.,\s]*'
31     max_distance_from_keyword: 2
32
33   address:
34     keywords:
35       - "dia chi"
36       - "address"
37       - "duong"
38       - "phuong"
39       - "quan"
40       - "huyen"
```

```
41     - "tỉnh"  
42     - "thành pho"  
43     - "tp"
```

Listing 8: Cấu hình luật cho phương pháp Baseline

D. Mẫu kết quả trích xuất

Bảng dưới đây trình bày một số mẫu kết quả trích xuất đại diện, so sánh giá trị dự đoán (Prediction) với nhãn thật (Ground Truth) trên tập kiểm tra.

Table D.1: Mẫu kết quả trích xuất so với nhãn thật

ID	Phương pháp	Trường	Ground Truth	Prediction	Đúng?
mcocr_001	DONUT	store_name	MINIMART ANAN	MINIMART ANAN	✓
mcocr_001	DONUT	date	2020-08-09	2020-08-09	✓
mcocr_001	DONUT	total	115000	115000	✓
mcocr_002	LAYOUTXLM	store_name	VINMART+	VINMART+	✓
mcocr_002	LAYOUTXLM	address	123 Nguyễn Trãi, HN	123 Nguyễn Trãi	×
mcocr_003	Baseline	total	250000	25000	×
mcocr_004	LAYOUTXLM	date	2020-12-25	2020-12-25 10:30	×
mcocr_005	Baseline	total	54000	5400	×
mcocr_006	LAYOUTXLM	address	Lô 12, Lĩnh Nam, Hoàng Mai, HN	Lô 12, Lĩnh Nam	×
mcocr_007	DONUT	total	82000	120000	×
mcocr_008	LAYOUTXLM	store_name	BÁCH HÓA XANH	BÁCH HÓA XANH	×
mcocr_009	Baseline	date	2021-01-10		×

E. Giao diện Demo Gradio

Giao diện ứng dụng Demo Gradio cung cấp một công cụ trực quan hóa giúp nhanh chóng thử nghiệm khả năng nhận dạng và trích xuất của ba mô hình đối với các ảnh biên lai ngẫu nhiên tải lên từ thiết bị cá nhân.

Ứng dụng demo được xây dựng bằng Gradio, cho phép người dùng tải ảnh biên lai lên và nhận kết quả trích xuất thông tin theo thời gian thực. Giao diện hỗ trợ chọn phương pháp (Baseline, DONUT, LAYOUTXLM) và hiển thị kết quả dưới dạng bảng cùng với hình ảnh bounding box.



Figure E.1: Giao diện ứng dụng demo Gradio cho trích xuất thông tin biên lai

E.1. Hướng dẫn cài đặt và chạy demo

Để cài đặt và khởi chạy ứng dụng demo, thực hiện các bước sau:

```
1 # Cai dat project o che do editable
2 pip install -e .
3
4 # Cai dat cac thu vien cho ung dung demo
5 pip install -r requirements/app.txt
6
7 # Khoi chay ung dung demo
```



```
8 python -m receipt_ie.app
```

Listing 9: Cài đặt và chạy ứng dụng Gradio demo

Sau khi chạy lệnh trên, ứng dụng sẽ khởi động tại địa chỉ `http://localhost:7860`. Người dùng có thể truy cập qua trình duyệt web để sử dụng giao diện demo.